Winter AI Study
Week 3: Sequence Data Learning

발표자 조현진

Sequence 데이터를 위한 모델의 발전 역사

이번 주차 스터디의 흐름

RNN ⟶ LSTM ⟶ Seq2Seq ⟶ Transformer

Sequence Data?

이전까지 공부했던 데이터

1 batch

Sample1: x
Sample2: y
Sample3: z
....

샘플 1개 당 관측 요소가 1개거나 차원이 있더라도 차원 간 순서가 없음

Sequence Data

1 batch

Sample1: x, y, z, ...
Sample2: a, b, c, ...
Sample3: q, w, e, r, ...

샘플 1개 당 여러 개의 관측 요소가 있고 이들 사이 순서가 존재

Sequence Data?

ex) 시계열 데이터(여러 timestep이 열을 이루고 있는 정형 데이터)

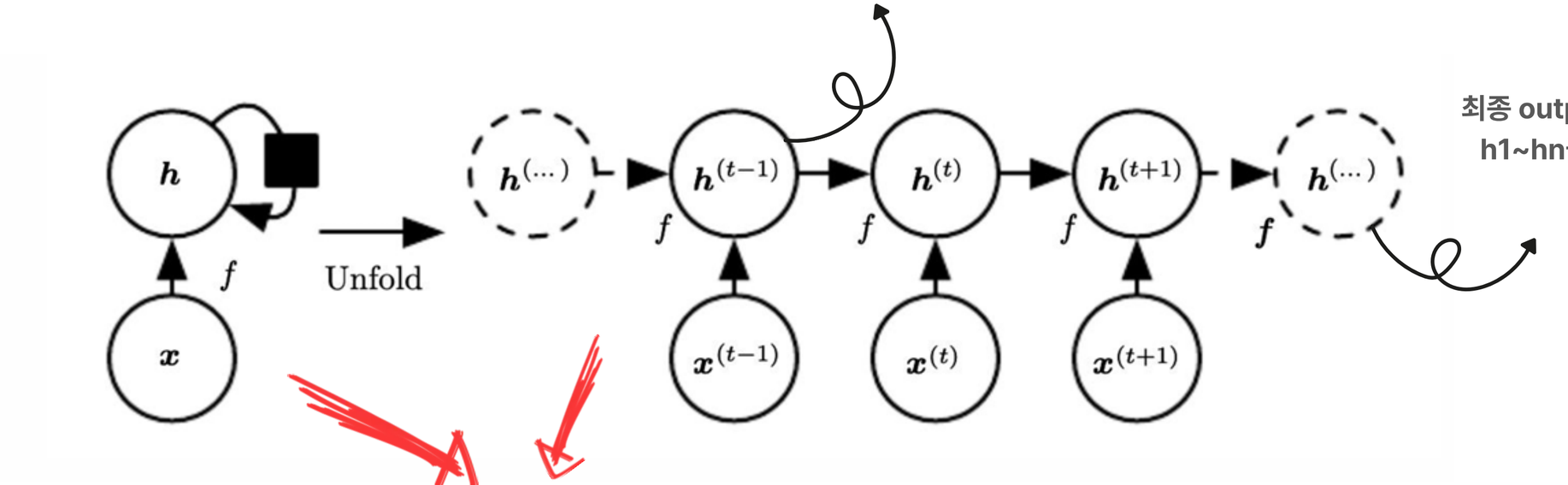
자연어 문장 (1문장은 여러 토큰의 sequence로 이루어짐)

Protein Sequence (한 개의 polypeptide는 여러 amino acid의 sequence로 이루어짐)

RNN: Memory-based Model for Sequence Data

t-1번째 관측 요소가 모델에 들어오고 계산된 값 (은닉층 값)
t-1번째까지 이해된 정보를 담고 있으며 t번째 관측 요소 계산 때 같이 인풋으로 들어감

은닉층 값이 이전 정보를 기억하는 일종의 메모리



최종 output에 h1을 쓰든, h1~h5를 쓰든, hn을 쓰든,
h1~hn을 쓰든, 은닉층 값을 MLP의 인풋으로 넣든,
정의한 task에 맞게 자유롭게 설계

x는 샘플 1개
(샘플 속 관측 요소들 간에 순서가 존재)

왼쪽의 구조가 결국
오른쪽 구조처럼 계산

RNN: Forward Pass 식 (은닉층 값 계산)

ht 구하는 식

이전 기억과 현재 input을 같이 넣어줌

$$h_t = f_W(h_{t-1}, x_t)$$

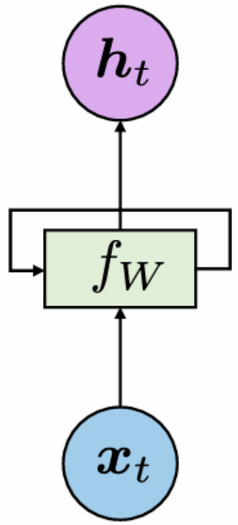
New state

Old state

Input vector at time step t

Function parameterized by learnable W

fw는 학습 파라미터 W에 대한 함수



fw

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

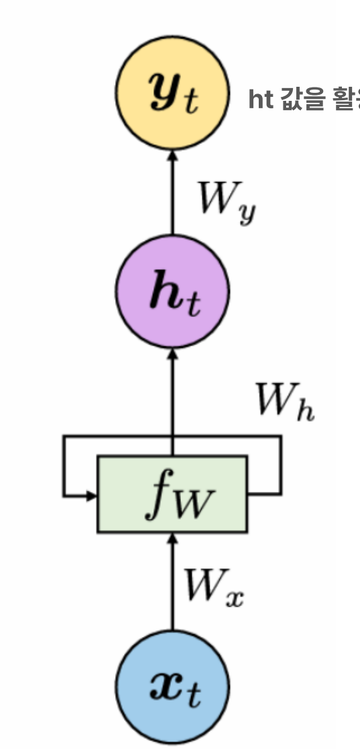
W: 학습 파라미터

ht-1: 바로 직전까지의 기억

xt: 현재 input

RNN: Forward Pass 식 (Output 계산)

최종 output에 h1을 쓰든, h1~h5를 쓰든, hn을 쓰든,
h1~hn을 쓰든, 은닉층 값을 MLP의 인풋으로 넣든,
정의한 task에 맞게 자유롭게 설계



ht 값을 활용해 우리의 task에 맞는 output y를 계산 (우리의 task에 맞게 설계 변형 가능)

$$y_t = W_y h_t$$

W: 학습 파라미터

RNN: Forward pass 계산

어떤 모델이든 차원 계산하는 것이 모델 이해에 중요 (모델 설계할 때에도 중요)

$$\boldsymbol{h}_t = \tanh(W_h \boldsymbol{h}_{t-1} + W_x \boldsymbol{x}_t)$$

h x 1 h x 1 d x 1

$$\mathbf{x} \in \mathbf{R}^{d \times 1}$$

$$\mathbf{h} \in \mathbf{R}^{h \times 1}$$

d는 인풋의 차원
한 배치 단위로 훈련하기 때문에 실제론 d x n의 형태이나 샘플 1개로 단순화 하였음
(n: 한 배치당 샘플 수)

h는 모델의 차원 (은닉층 값의 차원을 보통 모델의 차원이라 부름)

문제: 모델이 ht를 계산하게 하기 위해서, 우리는 모델을 만들 때 총 몇 개의 파라미터를 둬야할까?

Recap:

(m x n 행렬) * (n x d 행렬) = (m x d 행렬)

Sol) 필요한 파라미터를 묶어 놓은 Wh와 Wx가 몇 차원의 행렬이어야 하는지 파악해야 함

tanh와 같은 비선형 함수(활성화 함수)는 인풋의 차원을 절대 바꾸지 않음!!

저기서 +는 element-wise summation으로 대응되는 원소끼리 더하는 것이기 때문에 차원을 바꾸지 않음

최종 output은 h x 1 벡터이기 때문에, Wh는 h x h 행렬이어야 하고 Wx는 h x d이어야 함

따라서 설정해야 하는 학습 파라미터 개수는 $h^2 + hd$개이다.

(Wx 이런 선형 결합 형태의 계산을 취할 때, Wx라 표기하기도 xW라 표기하기도 하는데 어떻게 표기하냐에 따라 차원 표기도 순서가 바뀜!)

RNN: Forward pass 계산

어떤 모델이든 차원 계산하는 것이 모델 이해에 중요 (모델 설계할 때에도 중요)

$$y_t = W_y h_t$$

dy x 1 h x 1

$y \in R^{dy \times 1}$

dy은 아웃풋 차원

$h \in R^{h \times 1}$

h는 모델의 차원 (은닉층 값의 차원을 보통 모델의 차원이라 부름)

문제: 모델이 yt를 계산하게 하기 위해서, 우리는 모델을 만들 때 총 몇 개의 파라미터를 둬야할까?

Sol) 필요한 파라미터를 묶어 놓은 Wy가 몇 차원의 행렬이어야 하는지 파악해야 함

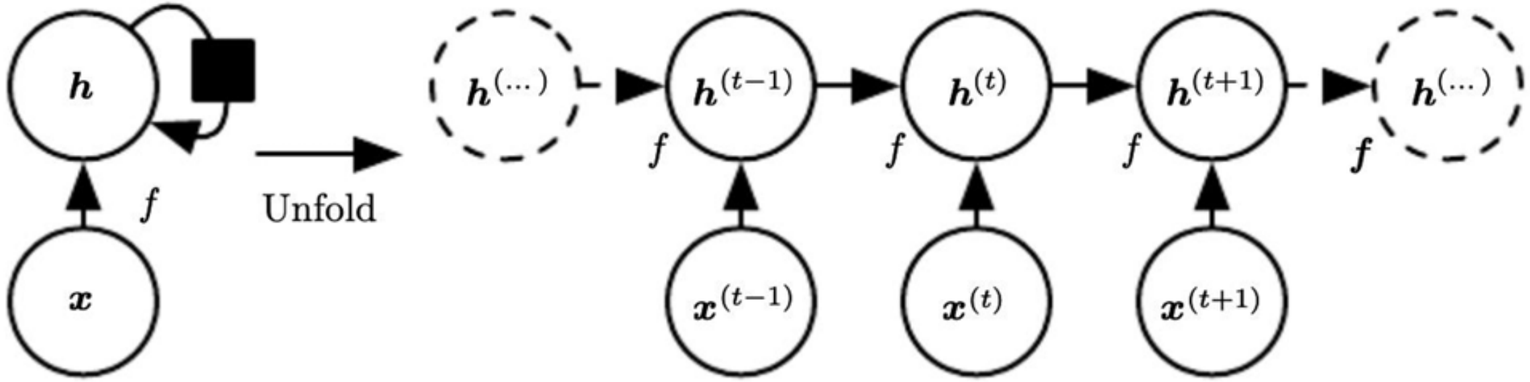
최종 output은 dy x 1 벡터이기 때문에, Wy는 dy x h 행렬이어야 함

따라서 설정해야 하는 학습 파라미터 개수는 hdy개이다.

Recap:

(m x n 행렬) * (n x d 행렬) = (m x d 행렬)

RNN: 구조적 특징



1. 왼쪽의 구조를 통해 알 수 있듯이 각 반복 요소(각 timestep)마다 **동일한 파라미터, 동일한 식으로** 은닉층 값을 계산함

2. 따라서 한 인풋 샘플 당 길이 제한은 없음, 그냥 반복 연산만 그 횟수만큼 해주면 됨

(여러 샘플을 한 번에 처리하는 배치 처리(훈련)를 할 때는 동일 길이로 맞춰 하나의 행렬로 다뤄야 하기 때문에 비어있는 자리는 0으로 채워야 함: 패딩)

ex. seq1 = [2, 3, 4]이고 seq2 = [1, 2, 3, 4, 5, 6]이라면...

훈련할 때 seq1과 seq2가 같은 배치에 들어갔다고 가정 → 이 둘을 묶어 하나의 행렬로 만들고 이걸 input 1개로 해서 모델에 넣어야 함

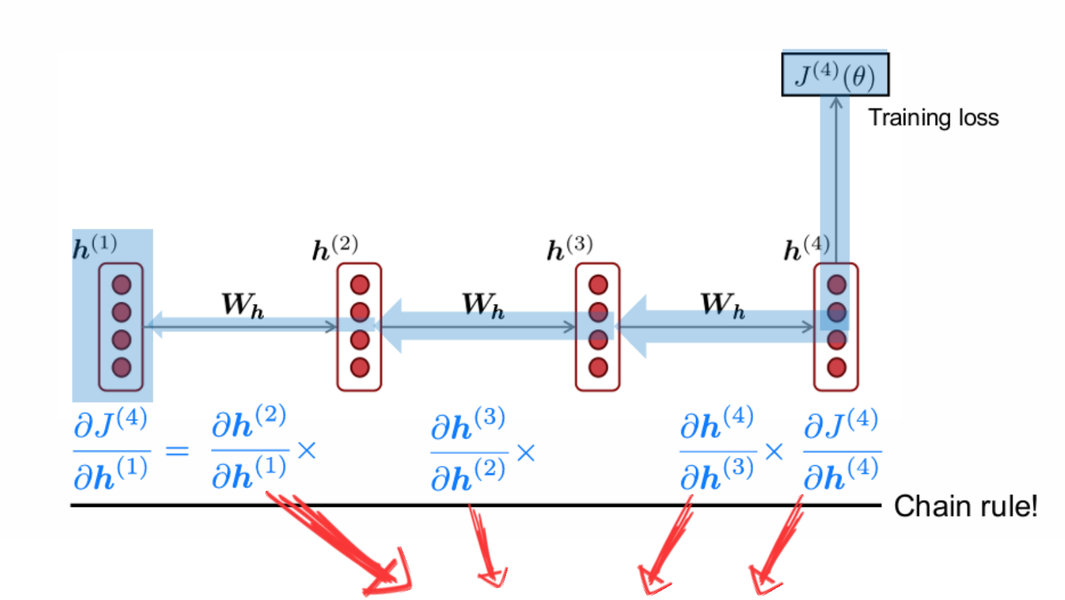
2 3 4 0 0

1 2 3 4 5

2 x 5 행렬

RNN: Backpropagation

RNN도 딥러닝 구조로서 뒤에서부터 앞으로, 계산된 activation gradient를 보내며 파라미터를 갱신하는데 사용



Activation 값이란?

이전 layer에서 다음 layer로 보내지는 값 (Forward pass 때, 2주차 참고)

Backpropagation도 이 activation에 대한 gradient가 뒤로 전달 되어야 함

뒤로 전해줘야 하는 activation gradient들 (경사 하강법 식에 직접 사용할 애들이 아님: 2주차 스터디 참고해주세요!)

경사하강법 식에 대입하는 gradient는 d(loss)/d(parameter)임!!

ex) d(loss)/dWh 계산

dJ/dWh = (dJ/dh4) * (dh4/dWh) + (dJ/dh3) * (dh3/dWh) + (dJ/dh2) * (dh2/dWh) + (dJ/dh1) * (dh1/dWh)

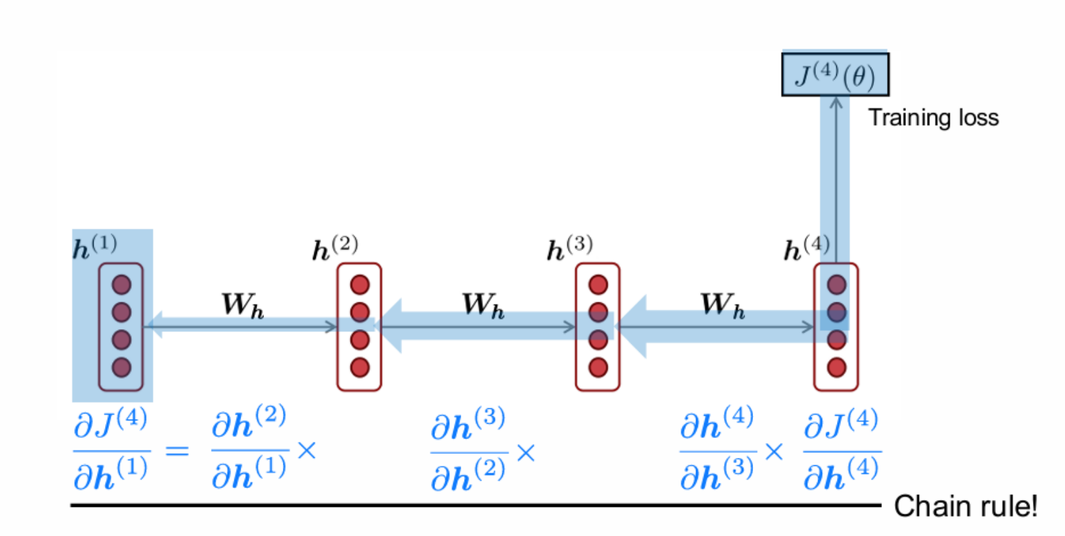
activation gradient / activation gradient / activation gradient / activation gradient

꼭 식이 어떻게 나왔는 지 이해해주세요 (2주차 스터디 참고해주셔도 됩니다.)

$$\boldsymbol{h}_t = \tanh(W_h \boldsymbol{h}_{t-1} + W_x \boldsymbol{x}_t)$$

computational graph 상에서 Wh는 h4에도, h3에도, h2에도, h1에도 값 계산에 사용됨 → 그럼 각 기울기들을 sum 해야 함(2주차 참고)

RNN: Vanishing Gradient



기여도 낮음

vanishing gradient가 발생하면

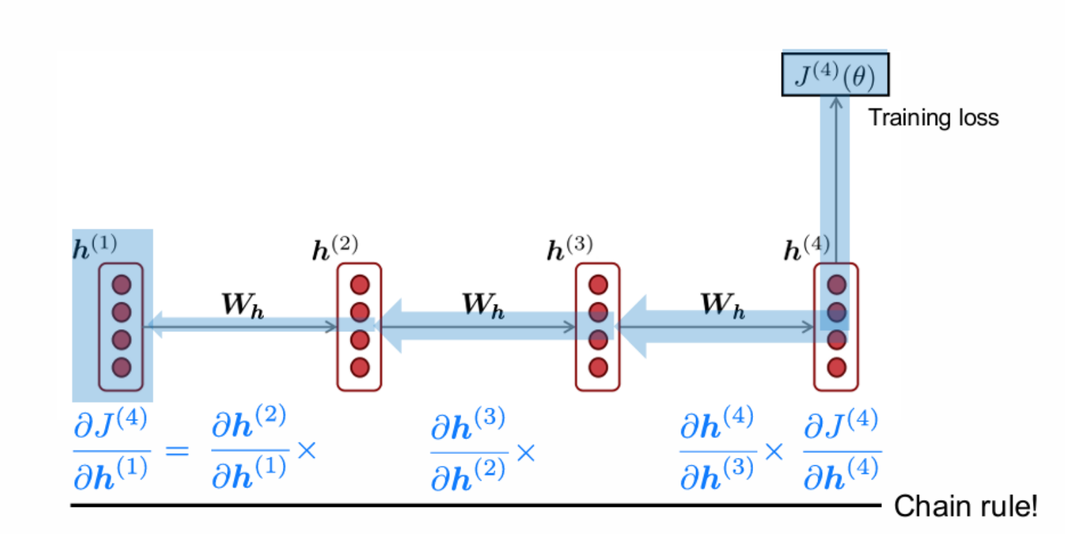
이 쪽 값이 작아짐

d(loss)/dWh = (dJ/dh4) * (dh4/dWh) + (dJ/dh3) * (dh3/dWh) + (dJ/dh2) * (dh2/dWh) + (dJ/dh1) * (dh1/dWh)

$\frac{\partial h^{(i+1)}}{\partial h^{(i)}}$ 가 너무 작으면??? ⇨ 앞쪽(앞쪽 timestep) gradient 값이 너무 작아짐 (by chain rule) ⇨ 앞쪽 timestep이 W를 업데이트 하는데 기여하는 정도가 점점 작아짐

== 점점 까먹음

RNN: Exploding Gradient



$\frac{\partial h^{(i+1)}}{\partial h^{(i)}}$ 가 너무 크면??? ⇨ 앞쪽(앞쪽 timestep) gradient 값이 너무 과도하게 증폭 ⇨ gradient는 파라미터 업데이트에 있어 크기가 중요한 게 아니고 방향이 중요

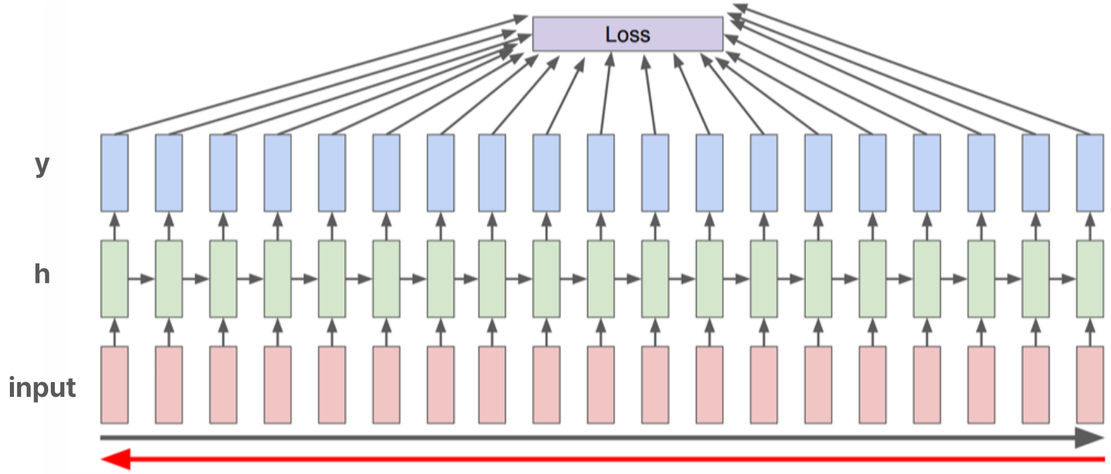
gradient 크기가 너무 크면 learning rate가 아무리 작아도 업데이트 되는

보폭이 너무 커져 optimum 값을 지나쳐 발산할 수 있음

Easy Solution: Gradient Clipping → Threshold를 하이퍼 파라미터로 설정해놔서 gradient가 threshold보다 큰 애는 threshold 값으로 크기를 잘라버림

RNN: Backpropagation (BPTT: Backpropagation through Time)

RNN backpropagation을 특히 BPTT라 부름

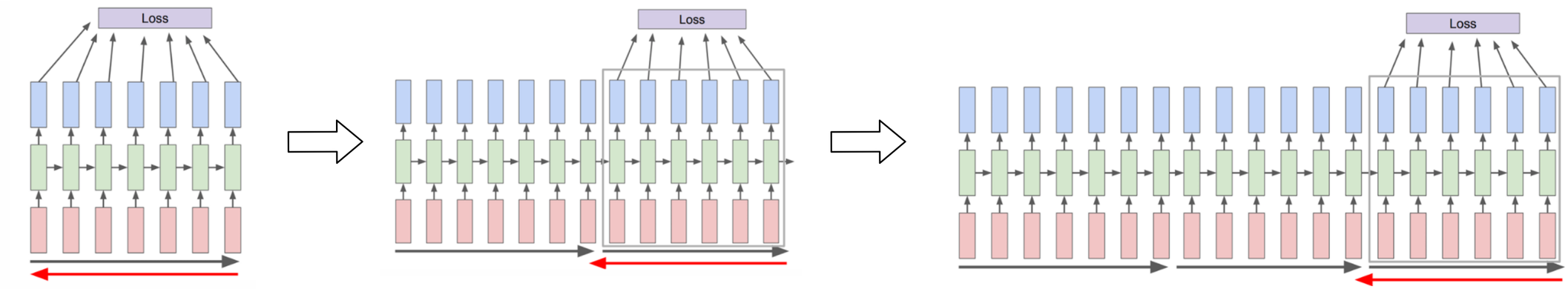


Case 1) 마지막 timestep forward pass가 끝나면 backpropagation 진행해서 첫 timestep까지 전달

Vanishing & Exploding Gradient 문제에 너무 취약해짐

RNN: Backpropagation (BPTT: Backpropagation through Time)

RNN backpropagation을 특히 BPTT라 부름

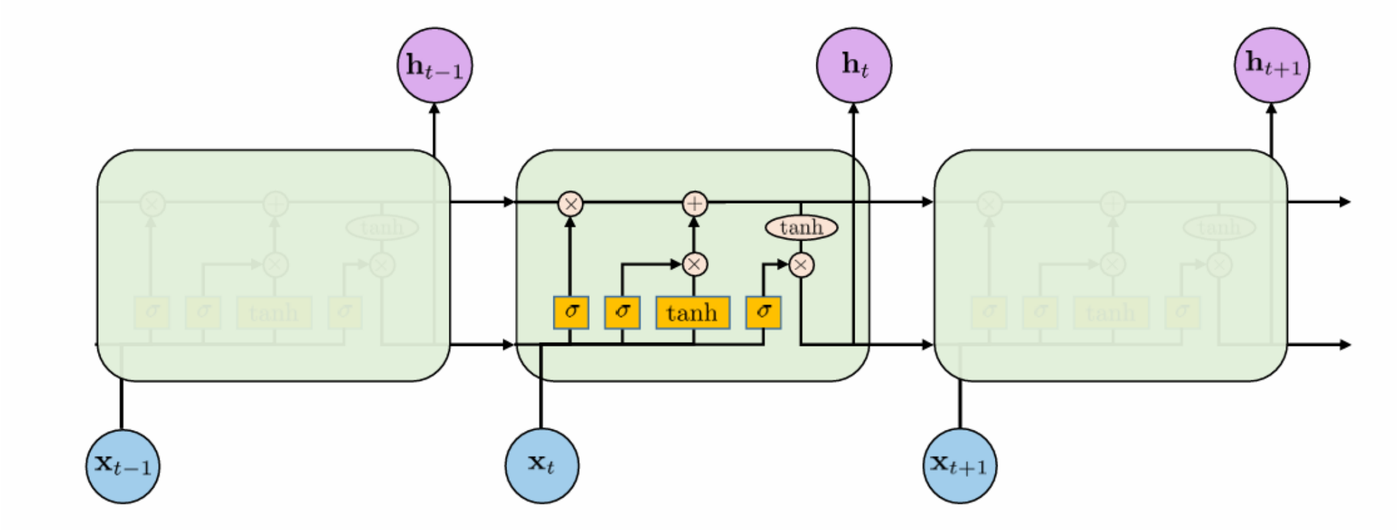


여전히 구간 안에서 vanishing & exploding gradient 문제 발생 가능

Case 2) Timestep을 끊어서 backpropagation 진행

LSTM: RNN에 memory 요소를 하나 더 추가

Long Short Term Memory



기존 RNN과 큰 구조는 동일

은닉층 값(h)을 계산하는 내부 구조가 바뀜

Cell state라는 요소를 추가하여 새로운 메모리 개념 구현

h가 단기 memory라면 Cell state는 장기 memory

내부에 3가지의 게이트를 둠

1. Forget Gate

2. Input Gate

3. Output Gate

LSTM: Forget Gate

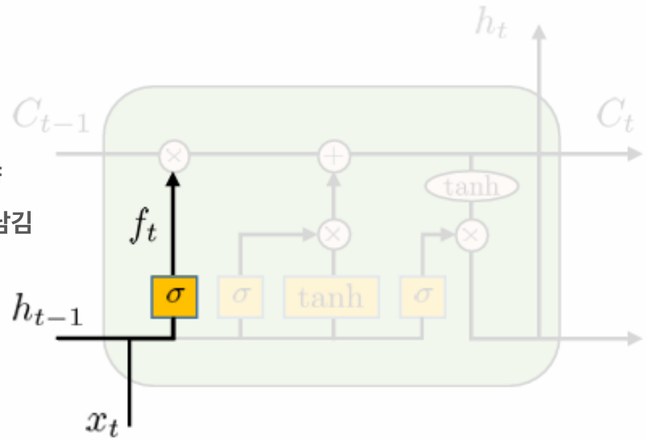
sigmoid 함수: 0~1 사이 값 반환(인풋이 클수록 1에 가까움)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

ft (0 < ft < 1)

기억하고 있는 요소 중 이 값을 곱해서 얼마만큼 지울거냐

ft * Ct-1 의 의미: 장기 메모리에 저장된 내용 중 ft만큼만 남김



Wf, bf: 학습 가능 파라미터

ht-1: 이전 timestep까지의 문맥(RNN과 동일)

xt: Timestep t에서의 인풋

[ht-1, xt]: ht-1과 xt를 concatenation

현재 인풋(xt)과 이전 단기 기억(ht-1)을 바탕으로 ,장기 메모리(Ct-1)에 저장된 내용 중 얼마만큼만 남길 것인지의 정도를 결정하는 단계

LSTM: Input Gate

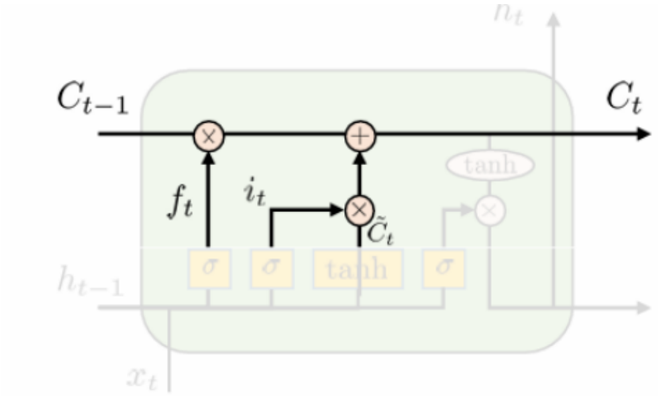
W, b: 학습 가능 파라미터

ht-1: 이전 timestep까지의 문맥(RNN과 동일)

xt: Timestep t에서의 인풋

[ht-1, xt]: ht-1과 xt를 concatenation

sigmoid 함수: 0~1 사이 값 반환(인풋이 클수록 1에 가까움)



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

이전 단기 메모리와 현재 인풋을 통해 생성될 content를 얼마만큼 반영할 것인가를 결정

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

이전 단기 메모리와 현재 인풋을 통해 계산된 content

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

이전 장기 메모리에서 잊을 건 잊고, 새로운 content를 it만큼 반영

사실상, 이 선형 결합이 LSTM이 만들어진 이유와 같음 (이유는 나중 슬라이드)

장기 메모리(Ct-1)에 ft를 곱해 잊을 건 잊어버리고, 이전 단기 기억과 현재 인풋을 바탕으로 계산된 새로운 content(C틸다)가 it만큼 반영되어 새로운 장기 메모리 Ct가 만들어짐

LSTM: Output Gate

W, b: 학습 가능 파라미터

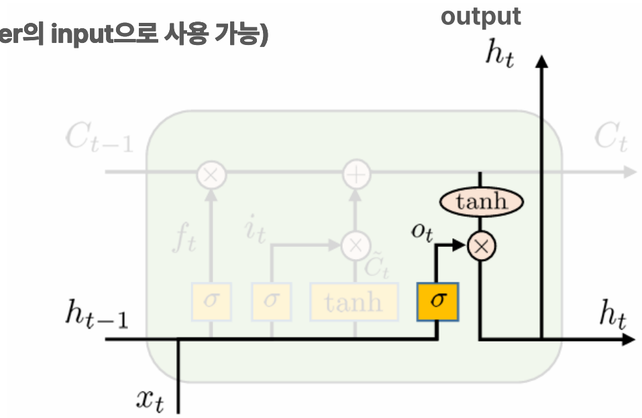
ht-1: 이전 timestep까지의 문맥(RNN과 동일)

xt: Timestep t에서의 인풋

[ht-1, xt]: ht-1과 xt를 concatenation

sigmoid 함수: 0~1 사이 값 반환(인풋이 클수록 1에 가까움)

ht는 단기 메모리이자 output(설계자에 따라 다음 layer의 input으로 사용 가능)



단기 메모리

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

전체 장기 메모리 내용 중 output(+ 단기메모리)으로 쓸 내용으로 얼마만큼을 사용할 것인가

$$h_t = o_t * \tanh(C_t)$$

장기 메모리 내용 중 Ot만큼 뽑음

업데이트된 장기 메모리(Ct) 내용 중 일부만 output + 단기 메모리로 사용

LSTM: Computational Graph

b(bias parameter)는 생략

Forget gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ Input gate: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

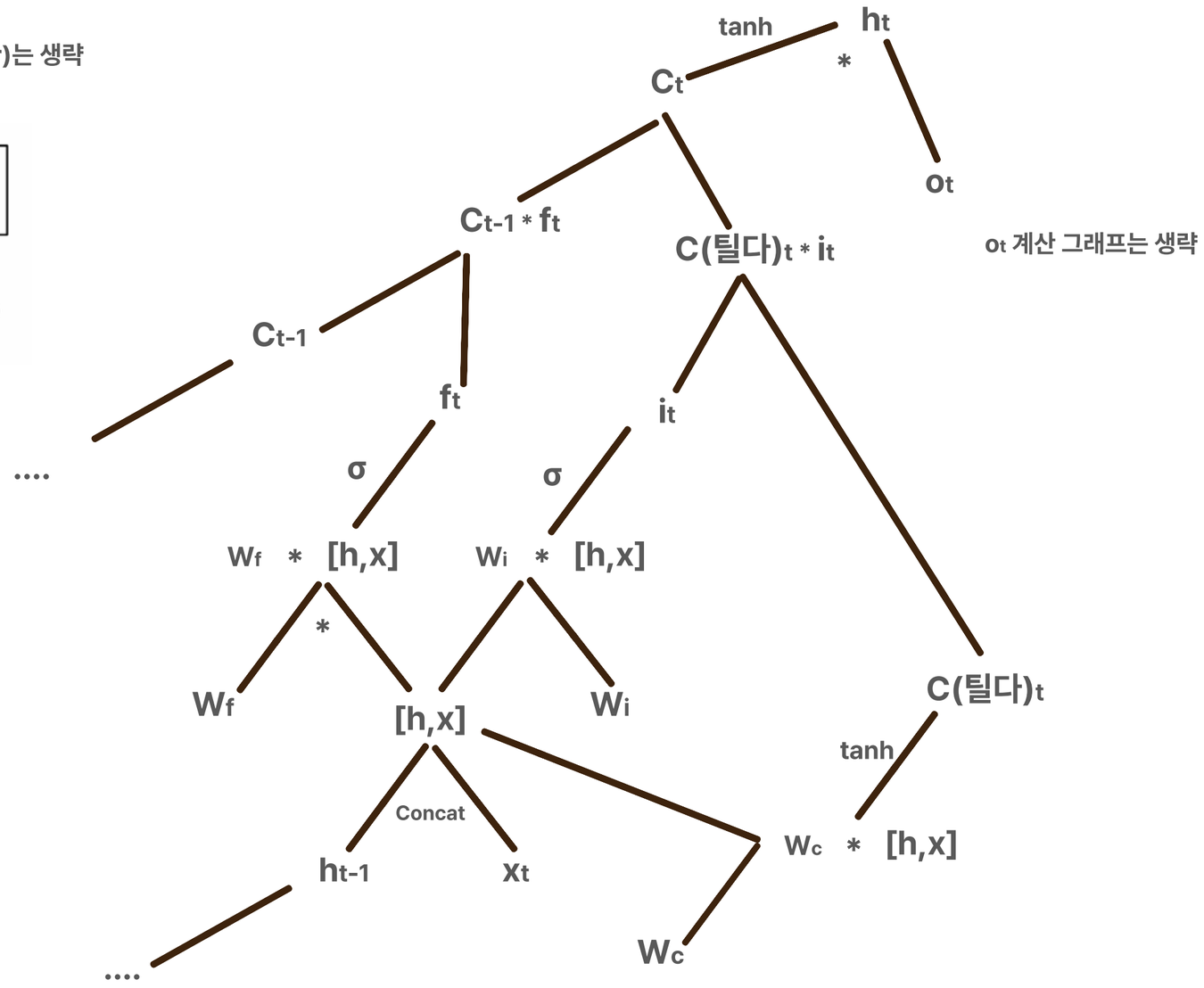
Previous cell state: C_{t-1} New cell content: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$

Updated cell state: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output gate: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

→ Hidden state: $h_t = o_t * \tanh(C_t)$

Forward pass 요약



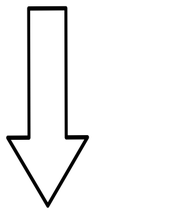
LSTM: 구조적 특징

Backpropagation할 때 d(loss)/d(parameter)를 계산하기 위해

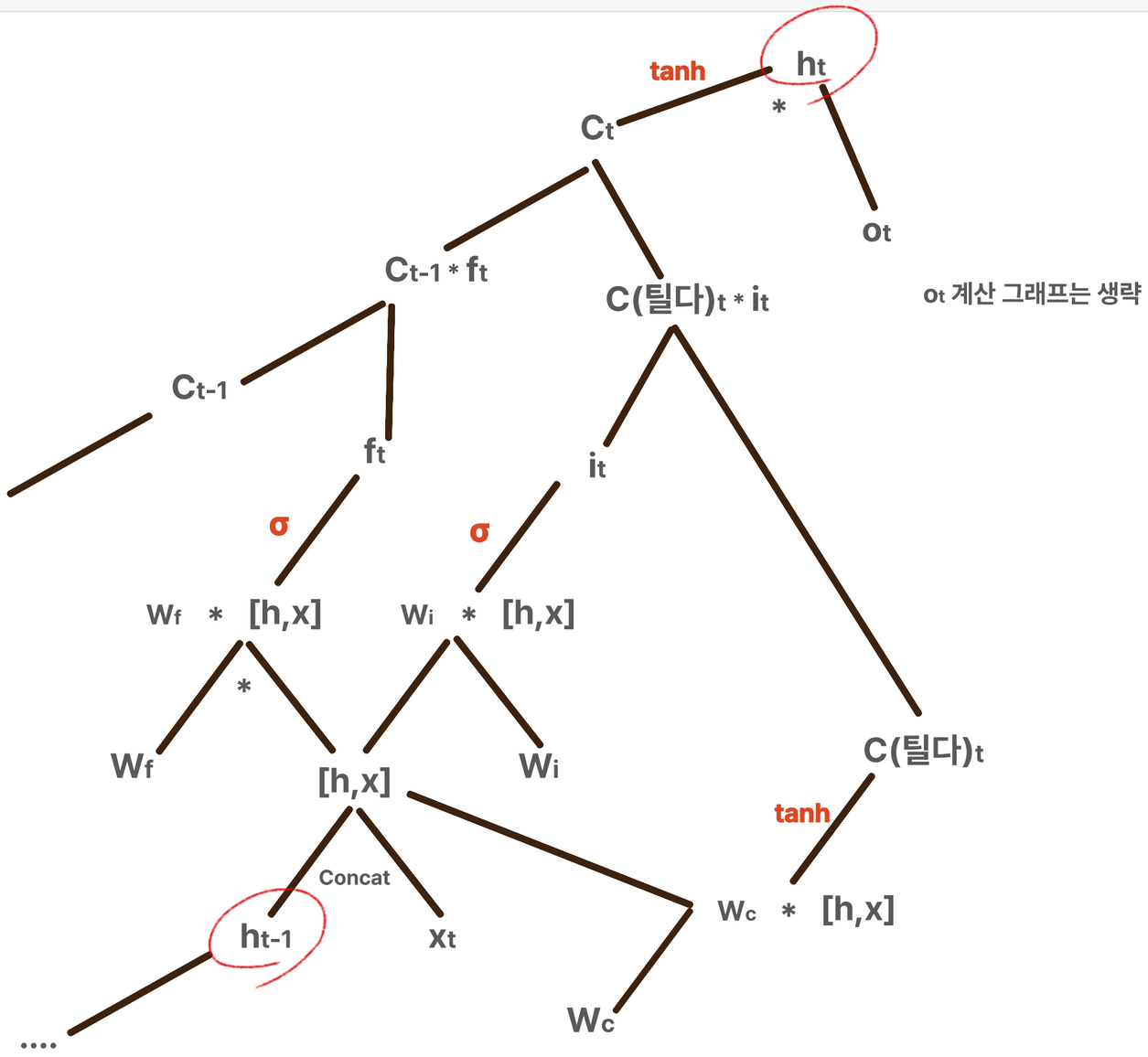
activation gradient인 d(loss)/dh를 뒤에서 앞으로 전달해야 함

Computational graph에서 젤 꼭대기에서 h로 가는 경로를 유심히 보면... (ht → ht-1)

tanh, sigmoid를 많이 거쳐야 함 하지만...



활성화 함수(sigmoid, tanh, ...)는 vanishing gradient의 주범: gradient가 작게 나오는 게 빈번



ot 계산 그래프는 생략

LSTM: 구조적 특징

문제 상황 정리

d(loss)/d(parameter)를 계산하기 위해 d(loss)/dh를 계속 전파해야함

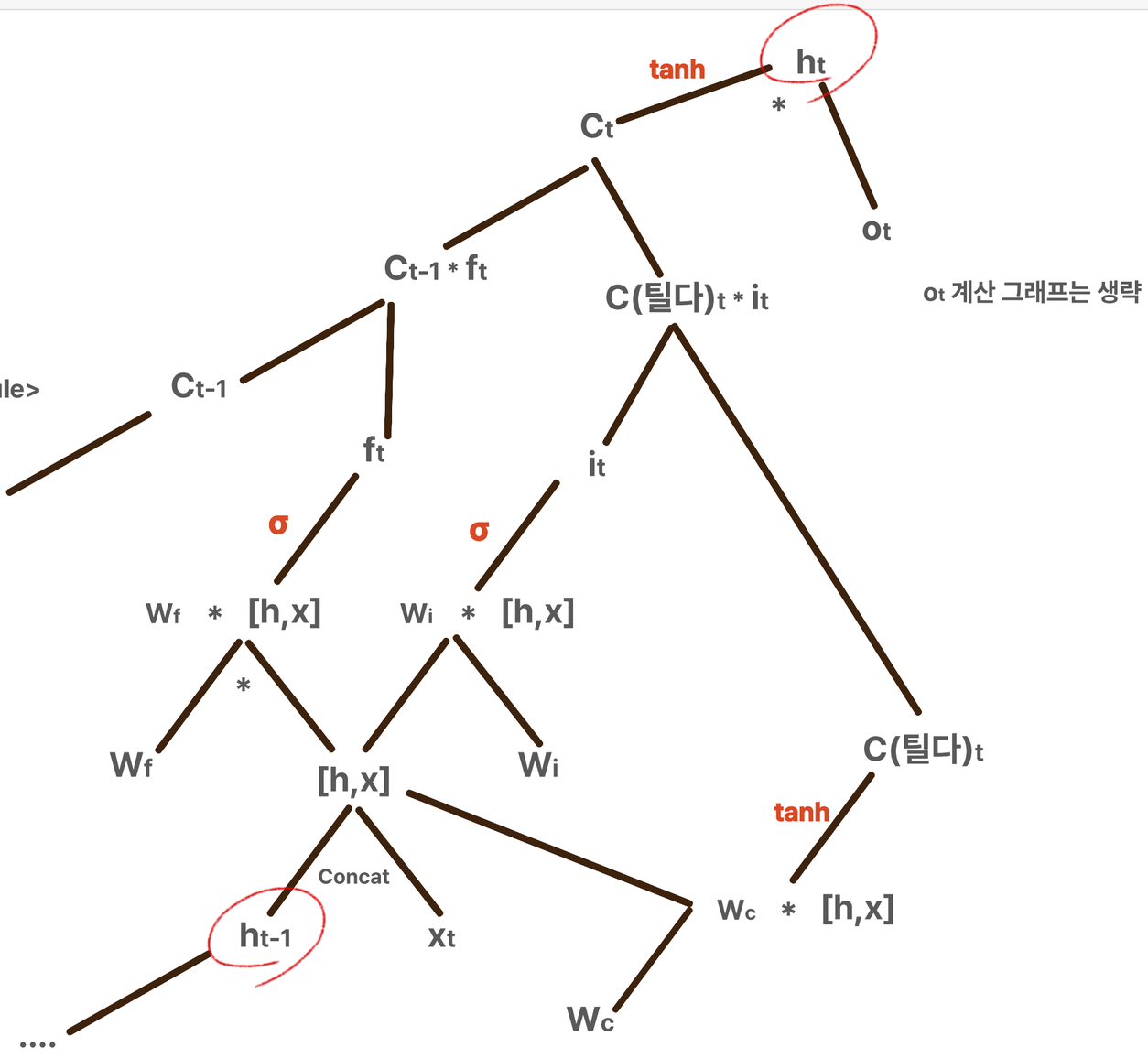
그럼 h0에서 파생되는 parameter의 gradient 계산할 때 사용할 activation grad는

$d(loss)/dh_0 = (d(loss)/dh_t) * (dh_t/dh_{t-1}) * \dots * (dh_1/dh_0)$ <chain rule>

computational graph 경로보면 활성화 함수가 많음

이 activation grad들 계산을 위해 또 chain rule로 쪼개면

vanishing gradient 문제 발생 확률 높음



ot 계산 그래프는 생략

LSTM: 구조적 특징

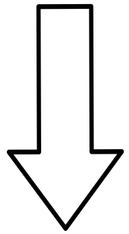
LSTM만의 대처 (RNN과 다른)

뒤에서 앞으로 전달하는 activation gradient로

d(loss)/dh를 쓰는 것이 아니라 d(loss)/dC를 씀

즉, 전파 경로를 단기 메모리를 타고 전파 시키는 것이 아니라

장기 메모리를 타고 전파 시킬 수 있도록 설계한 것



d(loss)/d(parameter)를 계산하기 위해 d(loss)/dC를 계속 전파해야함

그럼 C0에서 파생되는 parameter의 gradient 계산할 때 사용할 activation grad는

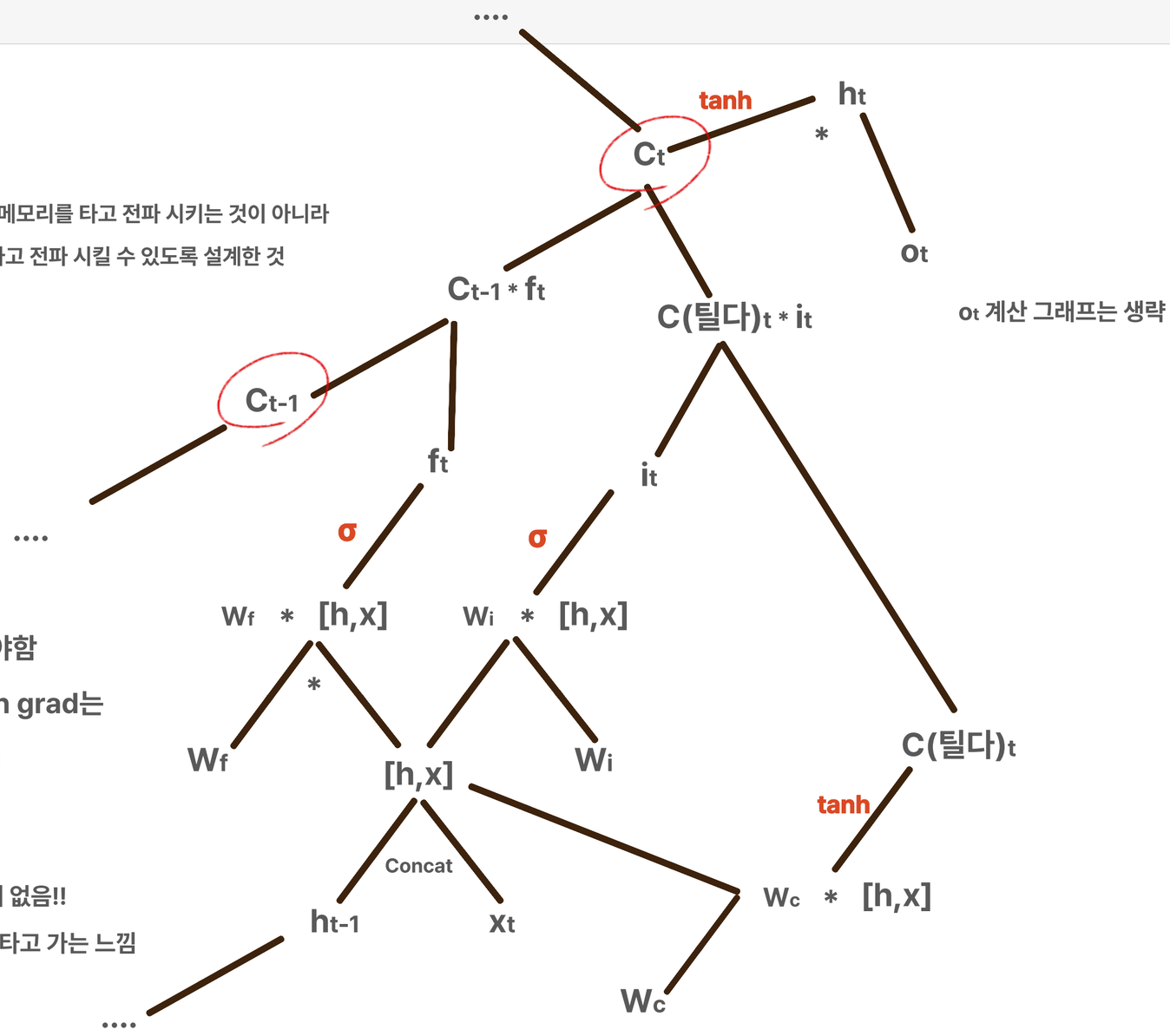
$$d(loss)/dC_0 = (d(loss)/dC_t) * (dC_t/dC_{t-1}) * \dots * (dC_1/dC_0)$$

경로에 선형 계산(활성화 함수 x)밖에 없음!!

h가 아닌 C를 통해 전파하면 마치 지름길을 타고 가는 느낌

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

사실상, 이 선형 결합이 LSTM이 만들어진 이유와 같음 (이유는 나중 슬라이드)



LSTM: 구조적 특징

d(loss)/d(parameter)를 계산하기 위해 d(loss)/dC를 계속 전파해야함

그럼 C0에서 파생되는 parameter의 gradient 계산할 때 사용할 activation grad는

$d(loss)/dC_0 = (d(loss)/dC_t) * (dC_t/dC_{t-1}) * \dots * (dC_1/dC_0)$

이런 LSTM의 장기 메모리와 관련된

구조적 특징을 잘 반영한 재밌는 해석

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$dC_t/dC_{t-1} = f_t$$

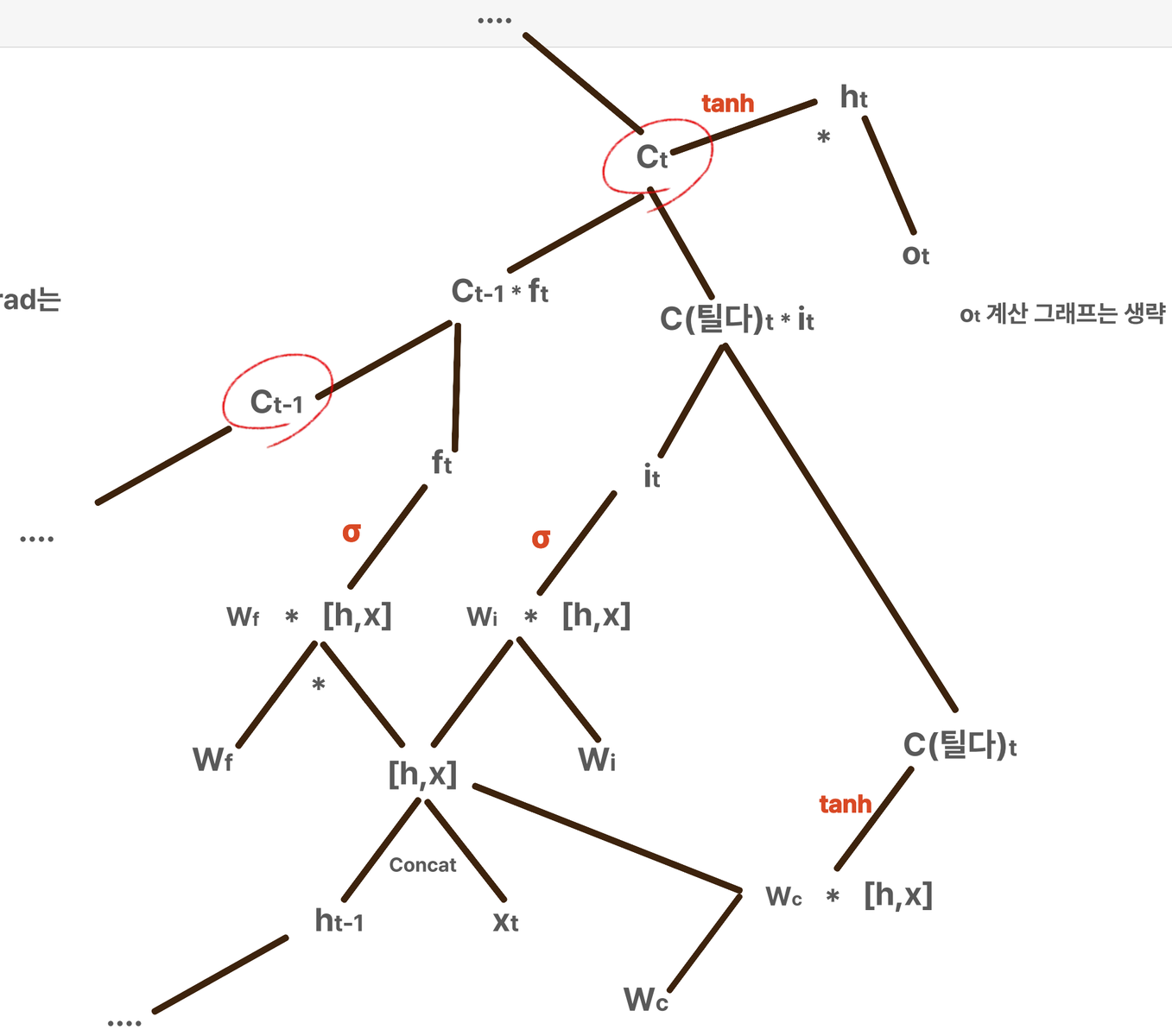
Timestep t일 때, f_t가 만약

1) 값이 크다면, 이전 장기 메모리 내용이 중요하니깐 최대한 많이 반영 해야하는 상황

→ Activation gradient도 큼 (정보를 많이 담고 있음)

2) 값이 작다면, 이전 장기 메모리 내용이 딱히 중요하지 않아서 많이 잊어도 됨

→ Activation gradient도 작음 (Vanishing gradient 허용)



GRU(참고만): LSTM의 아이디어를 똑같이 구현하였지만 좀 더 간단한 모델 (파라미터 절약)

1st step

Reset gate: $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$

Update gate: $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$

1st step or 2nd step

2nd step

New content: $\tilde{h_t} = \tanh(W \cdot [r_t * h_{t-1}, x_t])$

Hidden state: $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h_t}$

3rd step

Zt: 이전 메모리 값과 new content 중 어느 곳에 좀 더 많은 가중치를 둬 output 및 메모리 값을(ht) 생성할 지 결정

Bidirectional RNN or LSTM

input sequence가 주어졌을 때 앞 → 뒤로 가는 데이터 문맥도 중요하지만, 뒤 → 앞으로 가는 데이터 문맥도 task에 따라 중요할 수 있음

Hidden states

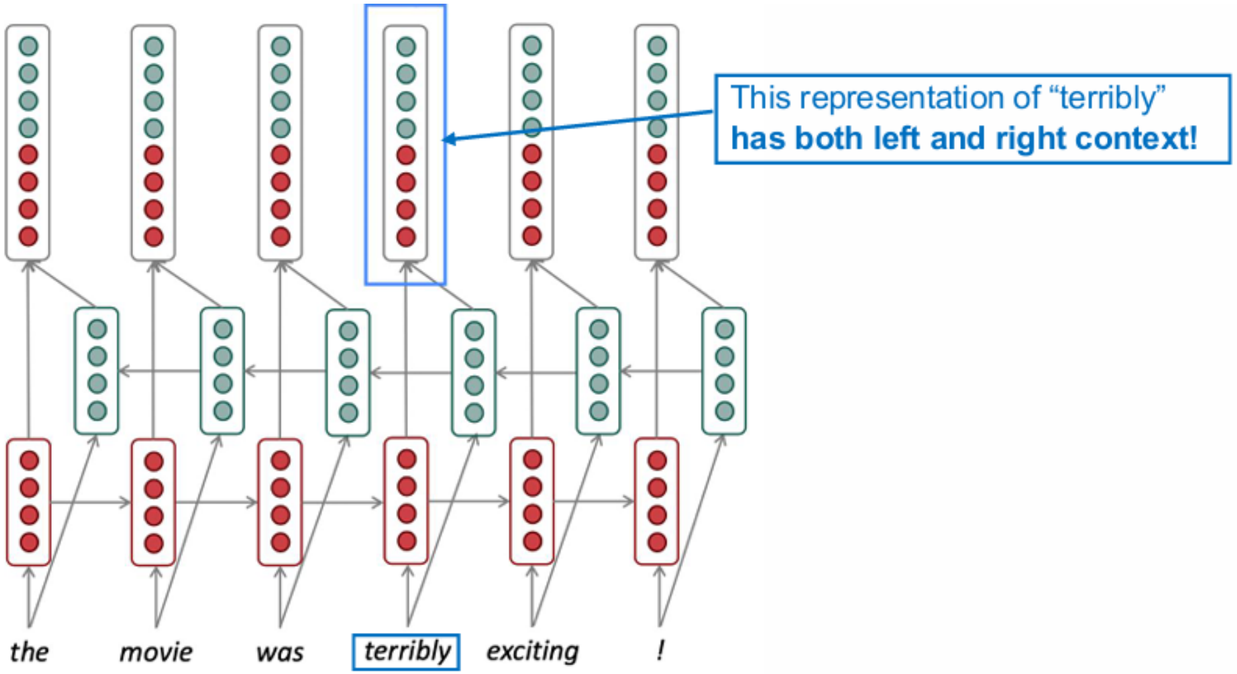
$$h^{(t)} = [\overrightarrow{h}^{(t)}; \overleftarrow{h}^{(t)}]$$

Backward RNN

$$\overleftarrow{h}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}\left(\overleftarrow{h}^{(t+1)}, x^{(t)}\right)$$

Forward RNN

$$\overrightarrow{h}^{(t)} = \mathrm{RNN}_{\mathrm{FW}}\left(\overrightarrow{h}^{(t-1)}, x^{(t)}\right)$$

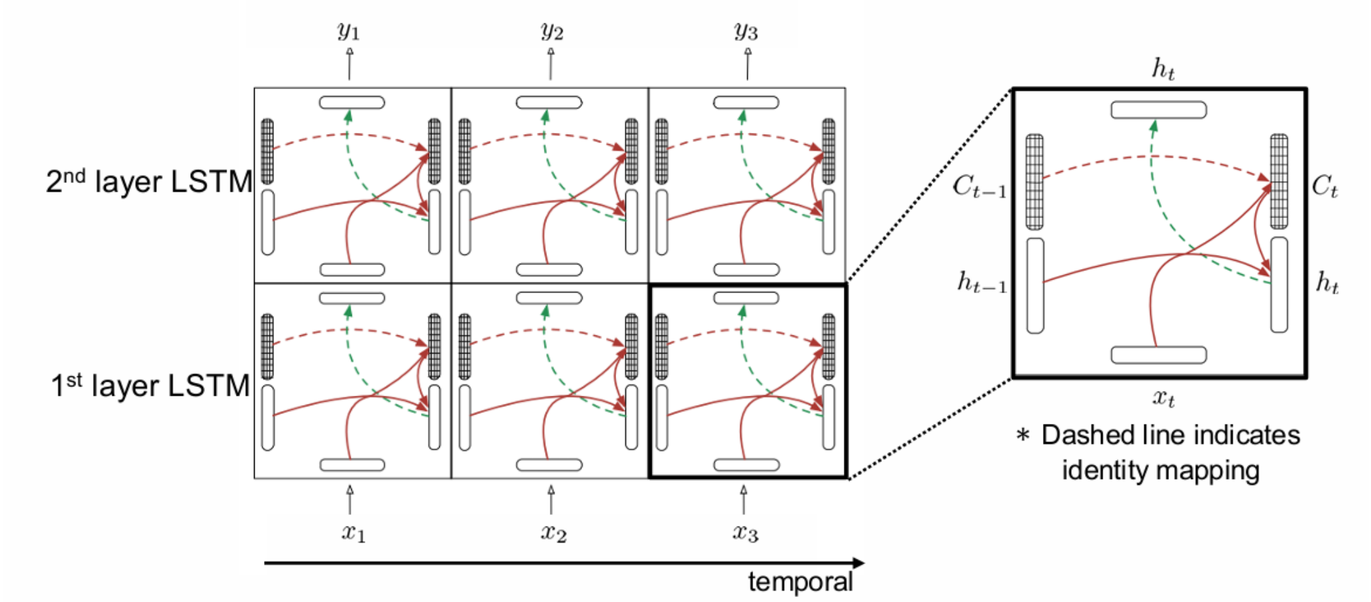


전체 input sequence를 알고 있을 때만 적용

앞 → 뒤로 읽는 기존 RNN(LSTM)을 두고, 뒤 → 앞으로 읽는 RNN(LSTM)도 두어 각각 결과낸 것을 concat

Concat된 애는 다음 layer의 input이 되거나 사용자 설계에 따라 다양하게 이용

Stacked(Multi-layer) RNN or LSTM



특정 time step t에서

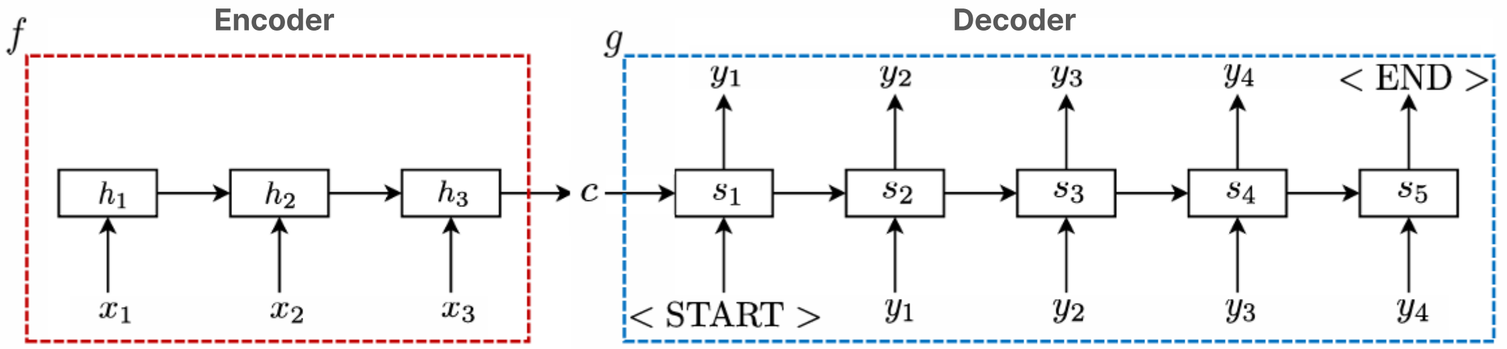
xt가 첫 번째 LSTM의 input으로 들어감 → 첫 번째 LSTM의 output이 두 번째 LSTM의 input으로 들어감 → 두 번째 LSTM의 output(yt)이 t time의 output

이후 Time step t+1로 이동

Seq2Seq: Encoder, Decoder의 개념을 sequence data에 처음 적용 (Transformer의 시초)

Sequence to Sequence: 시퀀스를 넣어서 시퀀스가 나온다

ex) NMT(Neural Machine Translation) task에 사용 (한국어 문장 시퀀스를 영어 문장 시퀀스로 변환)



Input sequence $\boldsymbol{x} = (x_1, x_2, x_3)$ and output sequence $\boldsymbol{y} = (y_1, y_2, y_3, y_4)$

Encoder: High level로 표현된 데이터로부터 feature를 추출

Decoder: 기계가 feature를 이해하고 다시 high level 형태로 변환

해당 feature는 모델 설계에 따라 우리가 추상적으로 해석 가능할 수도(interpretable)

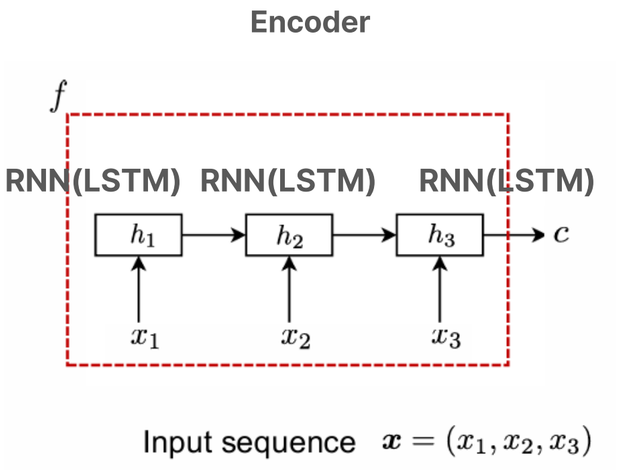
아니면 아예 이해하지 못하는 것일수도 있음

Seq2Seq은 RNN or LSTM을 시간 축 방향으로 연결하고 두 그룹으로 나눠 encoder와 decoder를 구현함

Sequence 데이터가 encoder를 통과하면 input sequence의 context를 함축하고 있는 벡터(c)로 변환

→ 이를 decoder가 받아서 context vector를 이해하고 우리가 원하는 내용의 새로운 sequence를 생성

Seq2Seq: Encoder

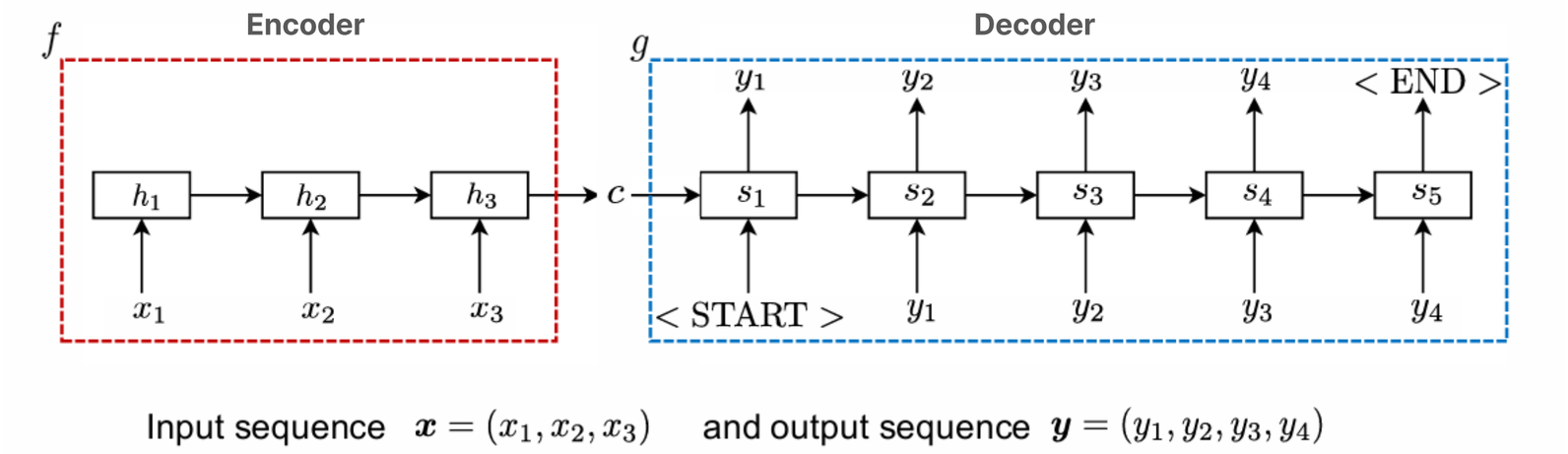


Context vector c: 인풋 시퀀스의 context를 담고 있는 벡터 (Encoder의 output)

x = [x1, …., xn]이라면

c는 h1~hn을 concat해서 만들수도 있고 설계자 마음임 (Original Seq2Seq은 그냥 마지막 은닉층 값인 hn을 그대로 c로 사용)

Seq2Seq: Decoder

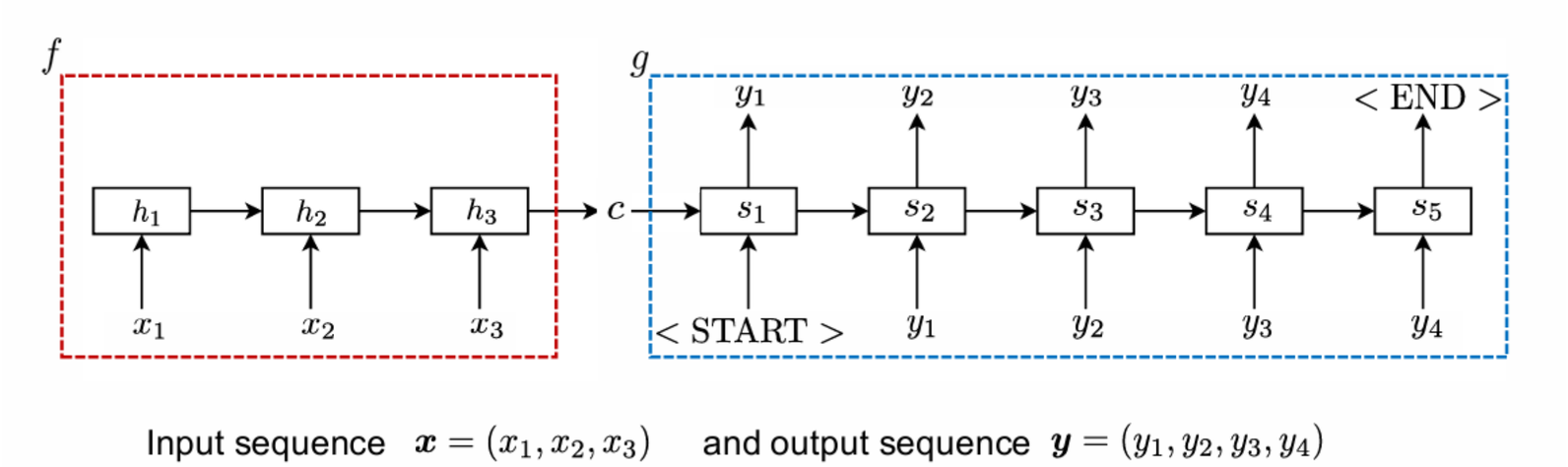


1st: Context 벡터와 <START>라는 special token을 받은 decoder의 첫 RNN(LSTM)은 내부에 hidden vector s1 생성하고 y1 output을 냄

2nd: 생성한 y1을 다시 input으로 넣음과 동시에 s1도 input으로 들어가 hidden vector s2를 생성하고 y2 output을 냄

3rd: <END> special token이 생성될 때까지 반복

Seq2Seq: Training

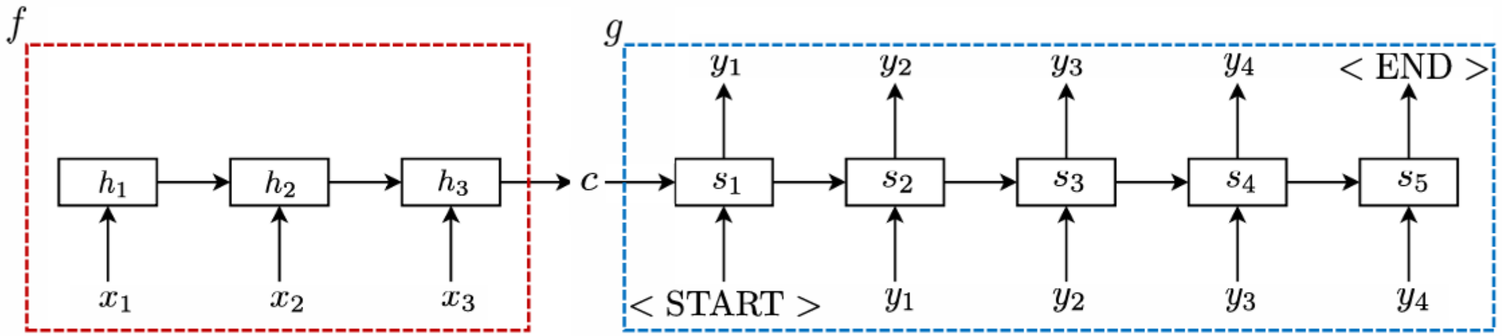


모델 Vocabulary를 미리 구축: Decoder가 생성할 토큰(y1, y2, y3, ...) 후보들을 미리 리스트업 해놔야 함 (Decoder가 아무거나 토큰으로 생성하는 것이 아님)

Decoder는 softmax 함수를 사용하여, (vocab_size) x 1의 벡터를 만들어 냄

해당 벡터는 vocab 속 각 토큰들이 해당 step의 output일 확률이 적혀있음 → Cross-Entropy (분류 task 일반적인 loss function, 1주차 Logistic Regression 참고)

Seq2Seq: Limitation



Input sequence $\boldsymbol{x} = (x_1, x_2, x_3)$ and output sequence $\boldsymbol{y} = (y_1, y_2, y_3, y_4)$

지금 decoder는 첫 step 때 context vector를 받아서 사실상 매 step마다 input sequence의 모든 토큰을 동일한 중요도로 참고해 output token을 생성함

하지만 매 step마다 중요하게 바라보아야 하는 input sequence 토큰이 달라져야함

Step1: "**I** love you" ⟶ "나는 너를 사랑해"
Step2: "I **love** you" ⟶ "나는 너를 사랑해"

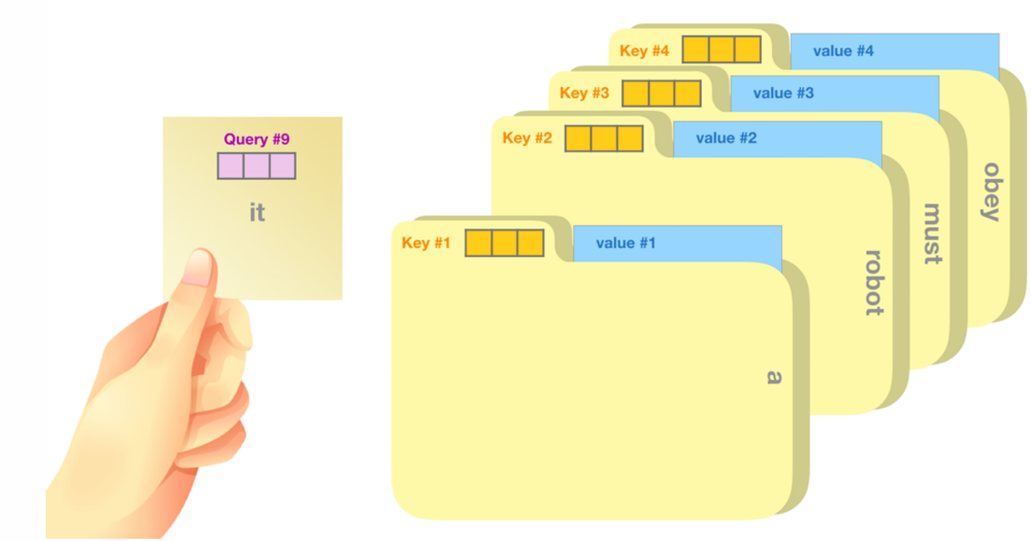
Machine Translation Task

Seq2Seq with Attention

Transformer의 핵심 메커니즘인 attention이 처음 도입

Attention

어떤 벡터가 있으면 그 벡터와 가장 관련성이 높은 벡터를 찾고 해당 벡터에 가중치를 부여해서 반영(즉, sequence의 어느 토큰에 focus를 해야 하는가를 결정)



Attention을 사용하려면 Query vector, Key vector, Value vector가 필요

1st: Query vector를 모든 key vector와 비교하며 검색하면서 관련도 점수를 계산

2nd: 각 Value vector에 계산된 관련도 점수를 가중치로 부여하고 summation

그럼 그 결과 "it"이라는 토큰과 나머지 다른 토큰들 사이의 관계 정보가 혼합된 벡터 하나를 얻게 됨

Seq2Seq with Attention

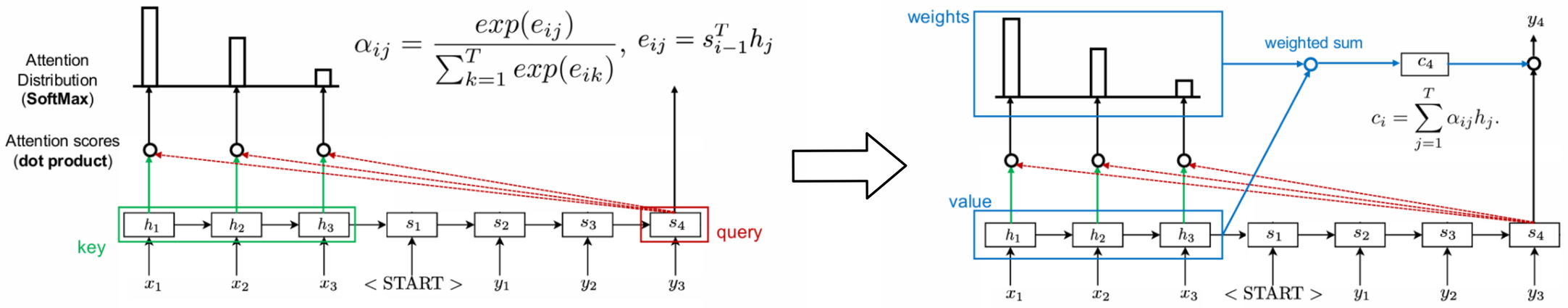
Transformer의 핵심 메커니즘인 attention이 처음 도입

Seq2Seq에서 사용한 attention

Decoder의 매 step마다 output y를 내기 전 hidden vector s를 query vector로 하여 input sequence의 각 토큰 key vector(h)에 대해 전부 검색 수행

각 input 토큰별 관련도 점수를 계산하고 각 토큰 value vector(h)에 가중치를 부여하여 summation함

Summation한 결과를 decoder가 output을 내기 전 반영



Seq2Seq with Attention 계산 예제

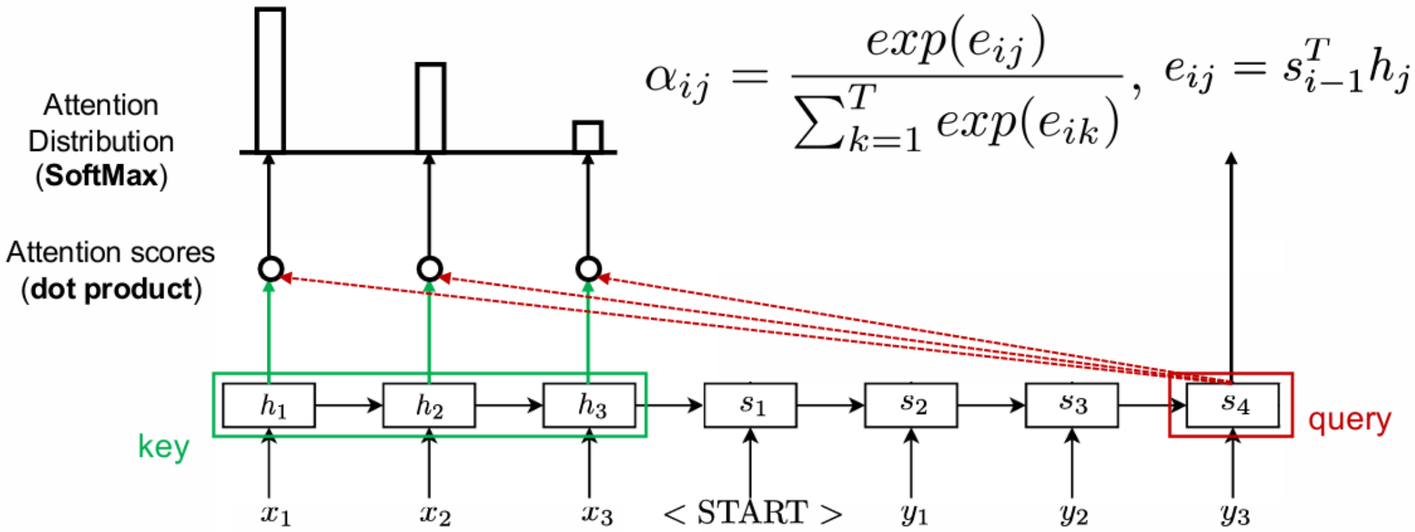
문제 조건 세팅

s4 = [1, 2, 3]

h1 = [5, 3, 1]

h2 = [2, 5, 3]

h3 = [0, 4, 9]



Decoder의 매 step마다 output y를 내기 전 hidden vector s를 query vector로 하여 input sequence의 각 토큰 key vector(h)에 대해 전부 검색 수행

검색 = 내적 (내적은 기본적으로 유사도 또는 관련도 정보를 포함, 값이 클수록 관련도가 높음)

e1 = s4 * h1 = 1 x 5 + 2 x 3 + 3 x 1 = 14

e2 = s4 * h2 = 1 x 2 + 2 x 5 + 3 x 3 = 21

e3 = s4 * h3 = 1 x 0 + 2 x 4 + 3 x 9 = 35

Softmax

α1 = exp(14) / (exp(14)+exp(21)+exp(35)) = 0.0000000008

α2 = exp(21) / (exp(14)+exp(21)+exp(35))= 0.0000008315

α3 = s4 * h3 = 1 x 0 + 2 x 4 + 3 x 9 = 35 = 0.9999991677 (아 s4 때는 x3에 특히 집중해야하는구나)

Seq2Seq with Attention

계산 예제

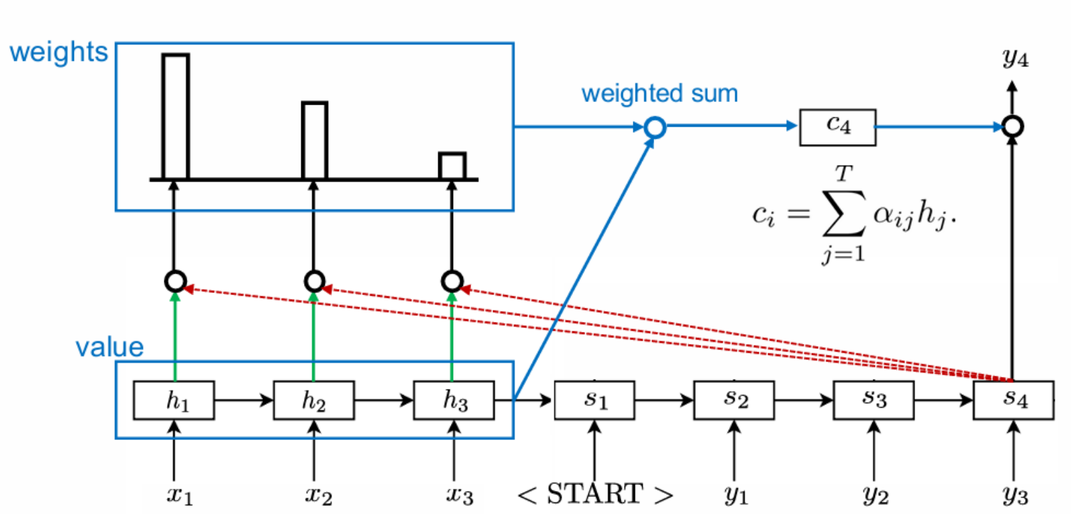
문제 조건 세팅

s4 = [1, 2, 3]

h1 = [5, 3, 1]

h2 = [2, 5, 3]

h3 = [0, 4, 9]



Softmax

각 토큰 value vector(h)에 관련도 점수를 가중치로 부여하여 summation함

Summation한 결과를 decoder가 output을 내기 전 반영

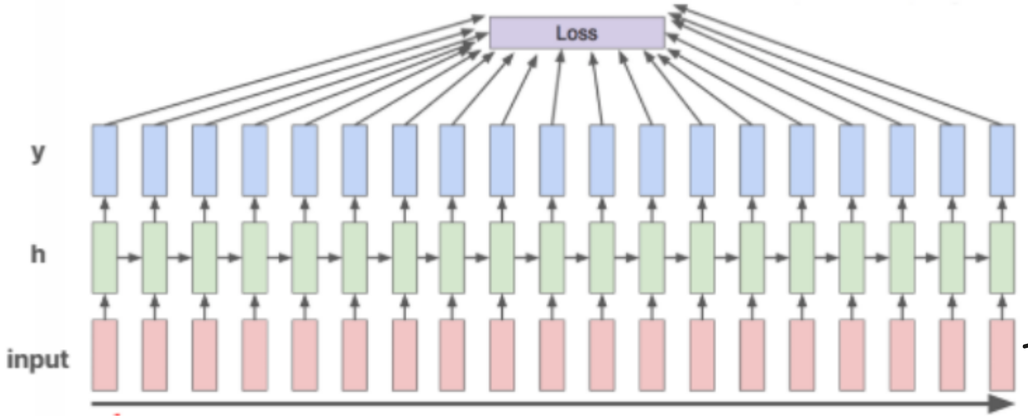
c4 = α1 * h1 + α2 * h2 + α3 * h3

= [0.000001667, 4.0000008307, 8.9999950046]

c4를 s4에 element-wise sum을 하든, 단순 concat을 하든 설계자 마음

그러고 추가 layer의 input으로 넣어 y4를 계산

Memory-based Sequence Model(RNN & LSTM)의 한계



여기에 있는 애랑 첫 번째에 있는 애 사이의 관계를 파악하기 위해서는 sequence length만큼 model을 돌려야했음

이슈 1: 관계를 제대로 파악하기에는 너무 거리가 먼 상황 (Vanishing G)

이슈 2: 병렬화가 안 됨 (뒤를 계산하기 위해서는 반드시 앞이 계산되어야 함)

반면, attention은 시퀀스 토큰들에 대해 한 번에 관계를 파악함

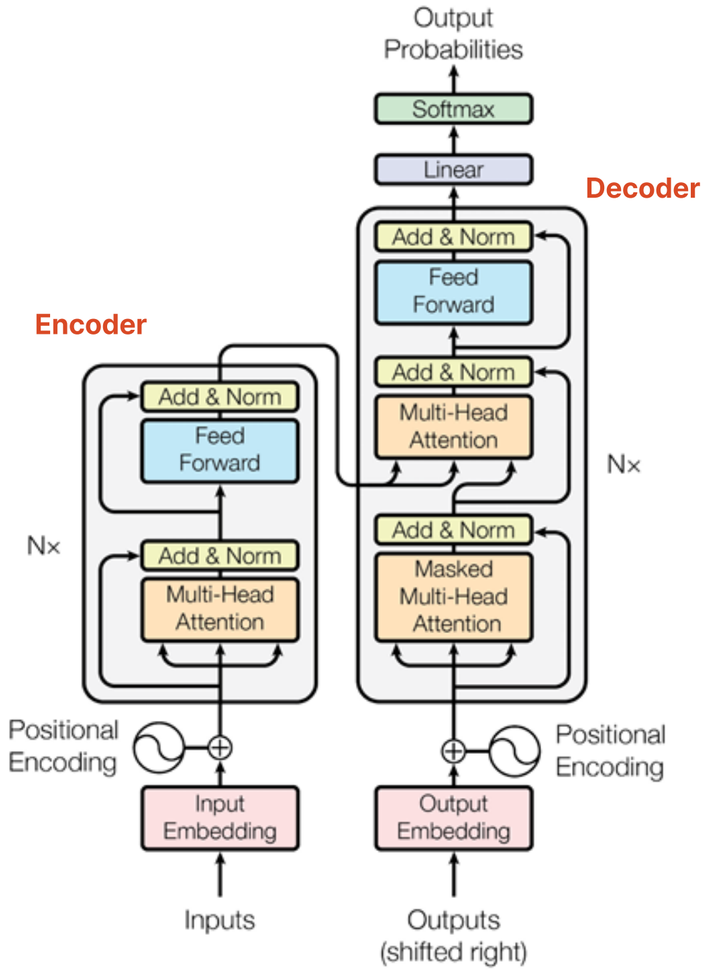
(앞에서 query를 key에 하나씩 검색해서 순차적으로 하는 것처럼 보이지만 실제론 행렬 하나로 묶어서 한 번에 계산함)

아, 그러면 sequence data를 attention만으로 처리해보자

→ Transformer

Transformer

언제 패러다임이 바뀔 지는 모르지만.. AI 분야에서 Transformer가 차지하는 비율이 매우 큼. 따라서 모델을 잘 이해하는 것이 중요...



Encoder-Decoder 구조

핵심 Layer

1. Embedding Layer + Positional Encoding

2. Multi-Head Attention Layer

3. Residual Connection + Layer Normalization

4. Feed Forward Layer

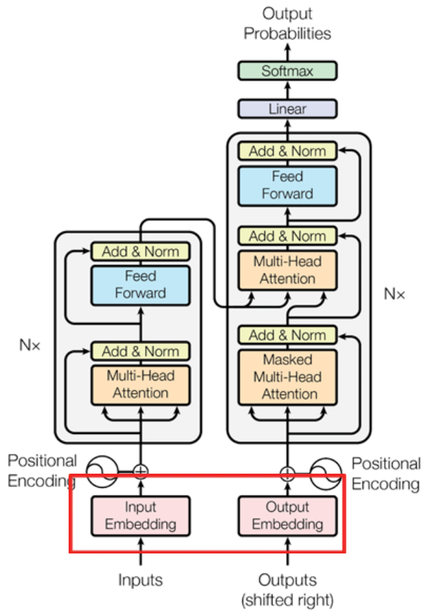
x = [x1, ..., xL]을 인풋 sequence라고 하면,

Encoder에서 1번 forward pass로 context vector 추출

Decoder에서 n번의 forward pass과정을 통해 y = [y1, ..., yn] sequence 생성

Transformer: Embedding Layer

인풋 토큰을 연산을 위한 실수형 벡터로 매핑




```
nn.Embedding(num_embeddings, embedding_dim)
```

(Vocab_size) x (Embedding 차원)

$$x_{fat} \times W_{V \times M} = e_{fat}$$

0	0	0	1	0	0	0

×

0.5	2.1	1.9	1.5	0.8
0.8	1.2	2.8	1.8	2.1
0.1	0.8	1.2	0.9	0.7
2.1	1.8	1.5	1.7	2.7

=

2.1	1.8	1.5	1.7	2.7

One-hot vector (해당 토큰이 lookup table 어느 위치에 있는지 표현)

실제 구현은 one-hot vector로 곱해주는 것이 아닌 그냥 단순 indexing

Lookup Table(임베딩 사전) 형태

임베딩 차원은 모델 차원(은닉 값 차원)과 같게

(Vocab_size) x (Embedding 차원)

$W \in R$

Training 할 때, 사용된 토큰에 해당하는 임베딩 값만

backpropagation으로 업데이트

인풋 토큰을 lookup table에서 검색

→ Embedding 값을 단순히 가지고 오는 형태

계속 강조하지만... 차원 이해하는 것은

모델 이해 및 설계에 매우 중요

Transformer: Embedding Layer

인풋 토큰을 연산을 위한 실수형 벡터로 매핑

x_{fat} × $W_{V\times M}$ = e_{fat}

0	0	0	1	0	0	0

×

0.5	2.1	1.9	1.5	0.8
0.8	1.2	2.8	1.8	2.1
0.1	0.8	1.2	0.9	0.7
2.1	1.8	1.5	1.7	2.7

=

2.1	1.8	1.5	1.7	2.7

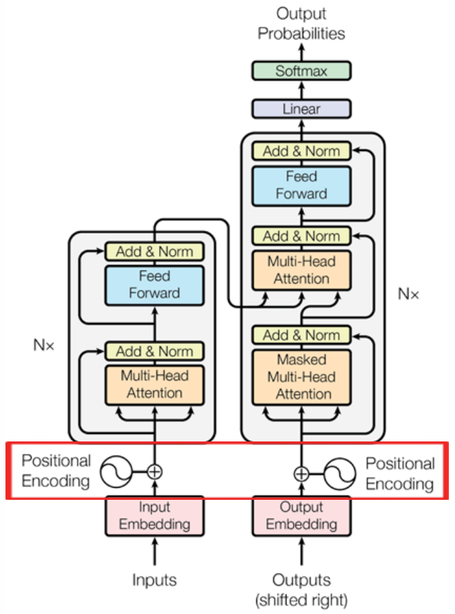
만약 한 샘플에 토큰이 L개라면

즉, input = [token1, ..., tokenL] $\in R^{L \times 1}$

인풋 임베딩 x는

x = [x1, ..., xL] $\in R^{L \times (\text{임베딩 차원})}$

Transformer: Positional Encoding



아직 각 토큰 임베딩에 위치 정보(time step)가 담기지 않음

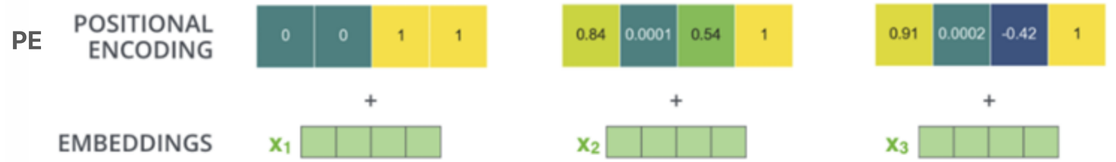
같은 토큰이어도 데이터 샘플 내에 어디에 위치해있느냐에 따라서도 임베딩 값이 달라지게 해야함

(만약, 다루고자 하는 sequence data에 순서는 중요하지 않다면 이 layer는 빼도 됨)

방법 1) Heuristic 방법(Original)

학습이 아닌 그냥 해당 식을 고정으로 두고 계산

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}}) \qquad PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



pos: 데이터 샘플 내 토큰 위치 번호

2i: 임베딩 짝수 번째 차원

2i + 1: 임베딩 홀수 번째 차원

dmodel: 모델 차원(은닉 값 차원)

방법 2) nn.Embedding

학습 가능한 lookup table을 두고 데이터 샘플 내 위치 번호에 따른 임베딩 값을 학습

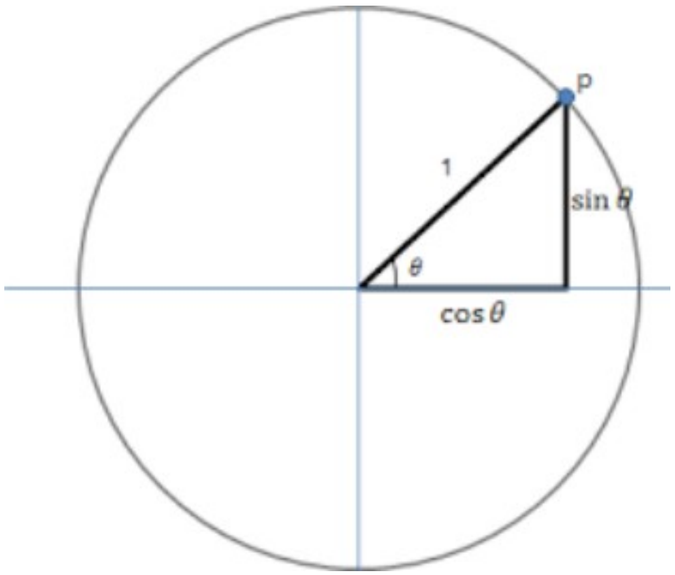
계산하고자 하는 토큰의 샘플 내 위치 번호를 통해 임베딩 값을 검색하고 토큰 임베딩에 더해줌

방법 3) Advanced 방법(RoPE,...)

관련 논문들 참고

Transformer: Positional Encoding (추가 슬라이드)

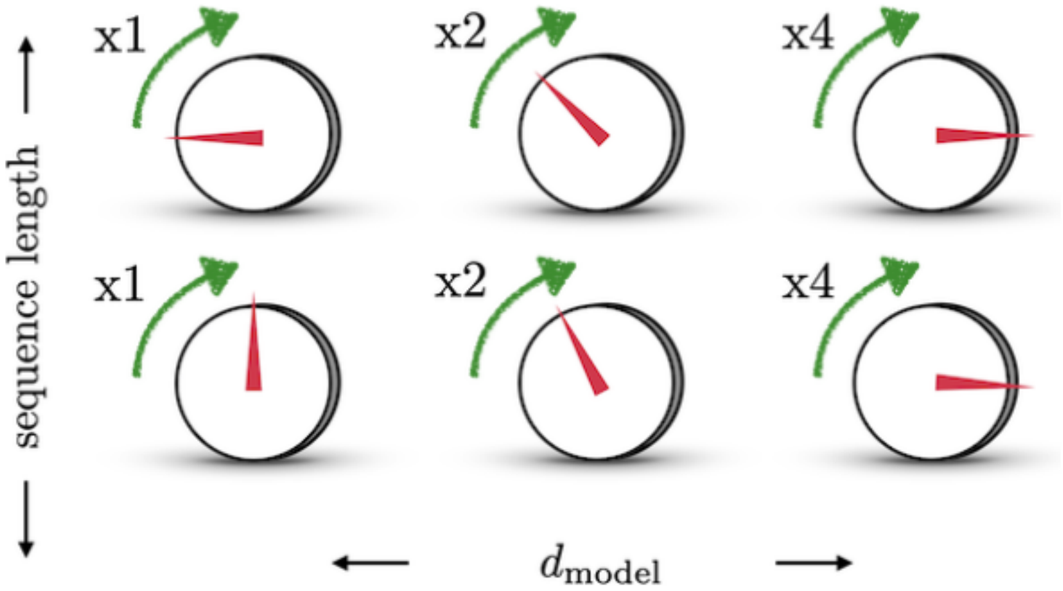
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}}) \qquad PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



2차원 평면 상에서 특정 point는 (cos, sin)으로 표현이 가능

Transformer: Positional Encoding (추가 슬라이드)

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}}) \qquad PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

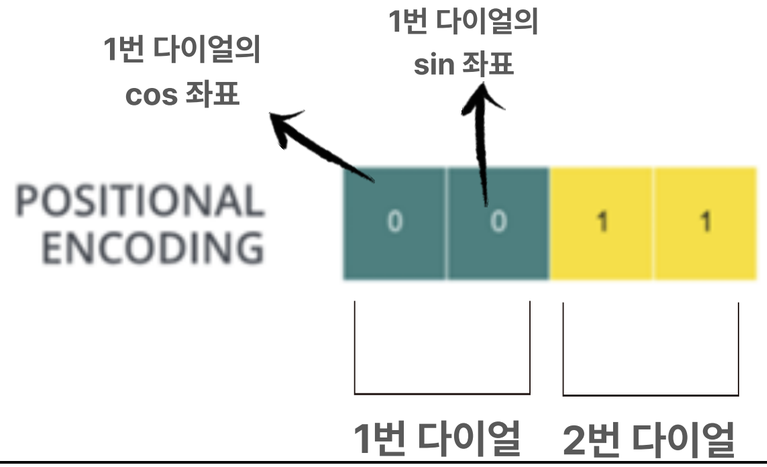


2차원 원으로 표현되는 다이얼이 있다고 생각

다음 토큰으로 넘어갈 때마다 해당 다이얼을 돌리면서 위치 feature를 조정한다고 생각해보자

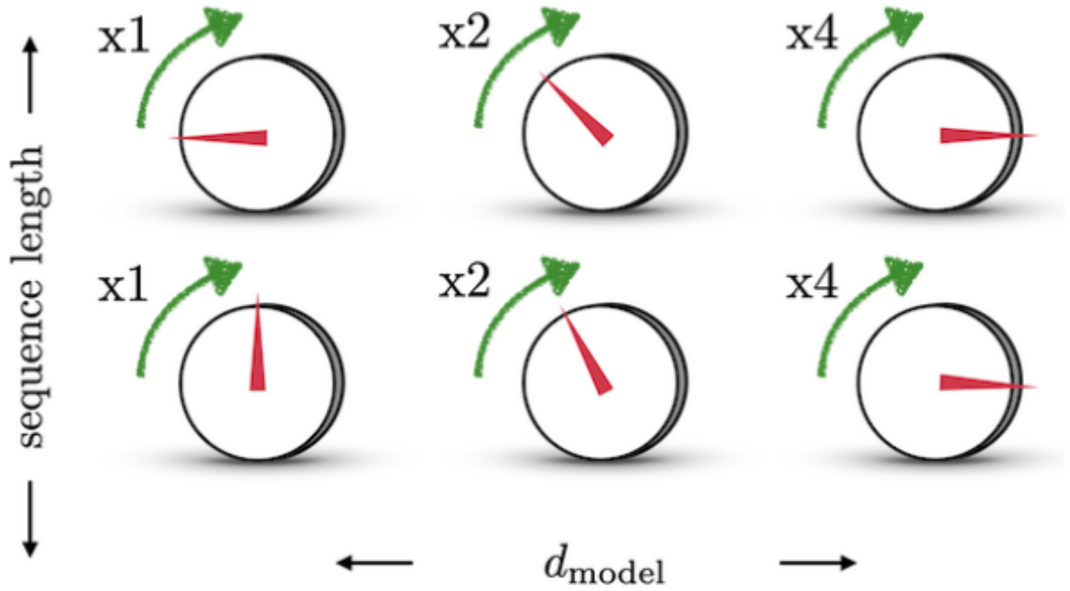
(각 다이얼은 위치 정보를 표현하기 위한 성분들, 해당 성분들의 조합으로 최종 위치 정보를 표현)

홀수는 왜 cos이고 짝수는 왜 sin인지에 대한 이유



Transformer: Positional Encoding (추가 슬라이드)

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}}) \qquad PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



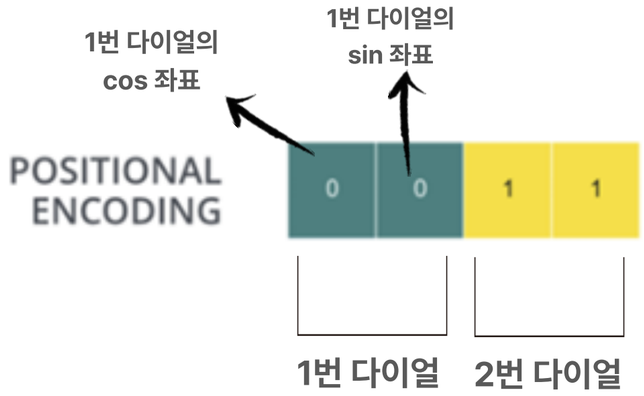
각 차원은 고유의 주기를 가짐: 각 다이얼은 한 번 이동할 때 서로 다른 보폭으로 돌림

같은 차원이라면 주기도 같은 동일한 sin or cos 함수 위의 점 (동일한 다이얼을 돌리는 것)

→ 특정 주기마다 같은 값을 가짐

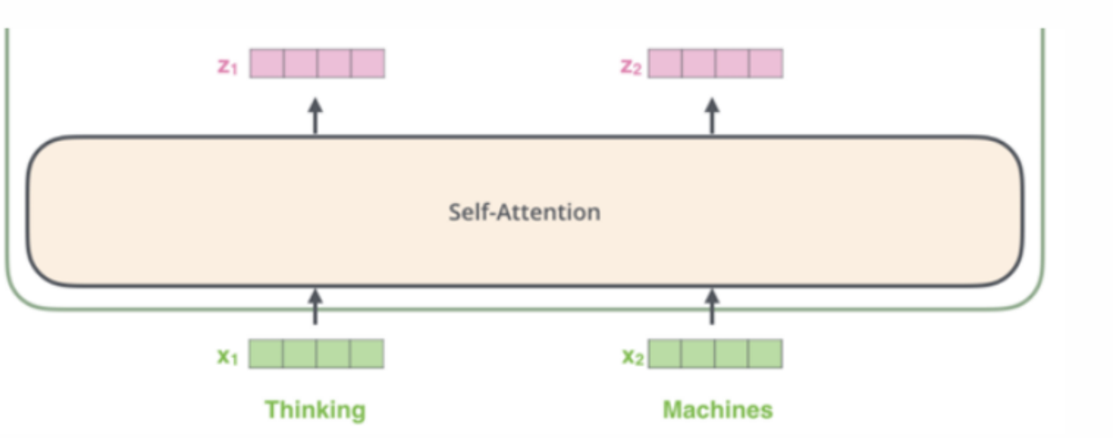
→ 해당 다이얼이 표현하는 위치 정보는 동일하지만 다른 다이얼이 표현하는 위치 정보가 다를 것임

→ 최대한 다양한 위치 정보를 학습 없이 표현할 수 있는 heuristic 방식



Transformer: Self-Attention

같은 sequence 샘플 내에서 각 토큰들 사이 서로 서로의 관계를 파악

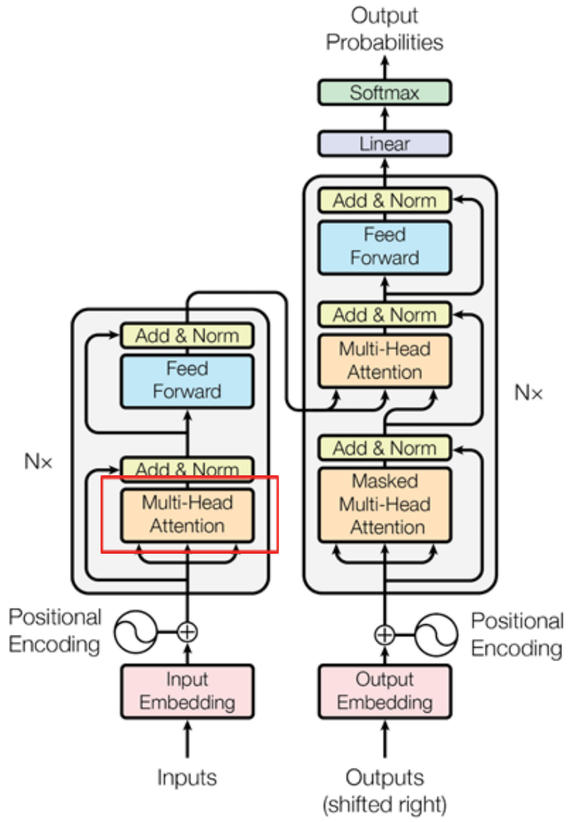


인풋 시퀀스가 x = [x1, ..., xL] 라면

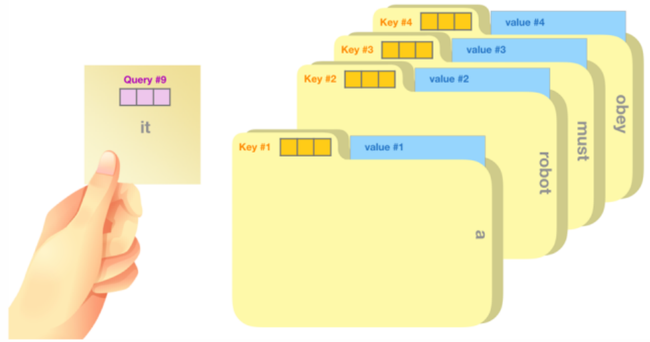
Self-Attention의 output은 z = [z1, ..., zL]

x1과 x1~xL 사이 관계가 반영된
x1의 임베딩 값

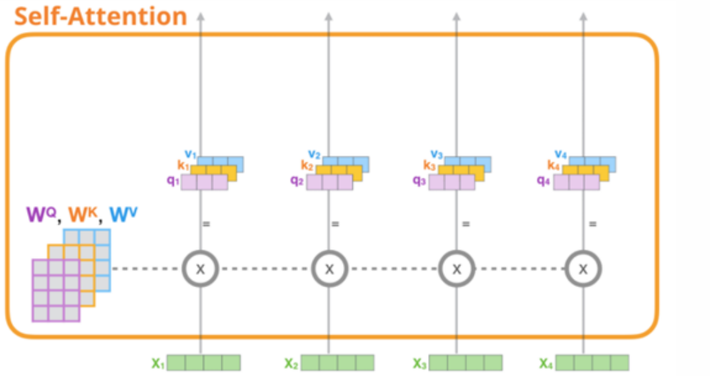
xL과 x1~xL 사이 관계가 반영된
xL의 임베딩 값



Transformer: Self-Attention



$$q_i = W^Q x_i, k_i = W^k x_i, v_i = W^V x_i$$

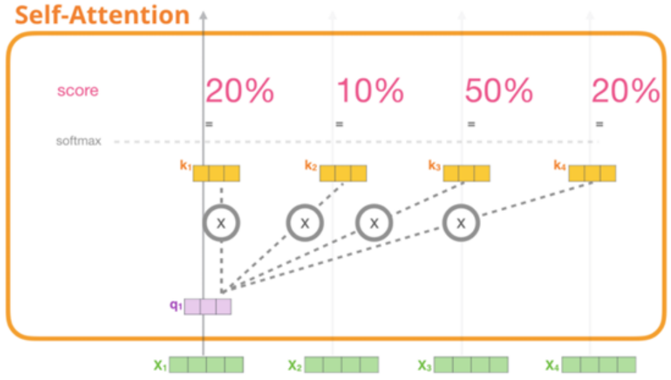


각 토큰 임베딩에 3개의 서로 다른 W(학습 파라미터)를 곱해주어 projection 시킴

→ 각 토큰별로 고유의 query vector, key vector, value vector가 생성됨

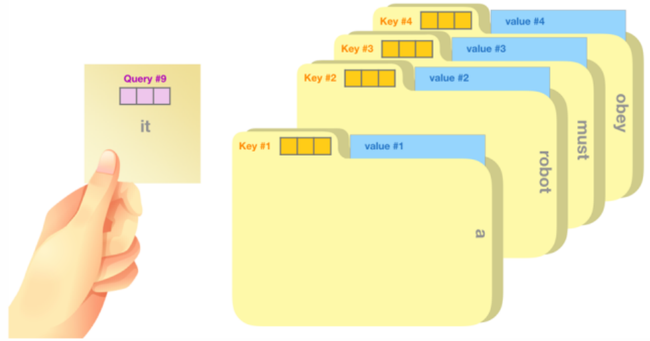
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})} \qquad e_{ij} = \frac{q_i^T k_j}{\sqrt{d}}$$

query와 key를 내적하고 **차원 제곱근으로 정규화**



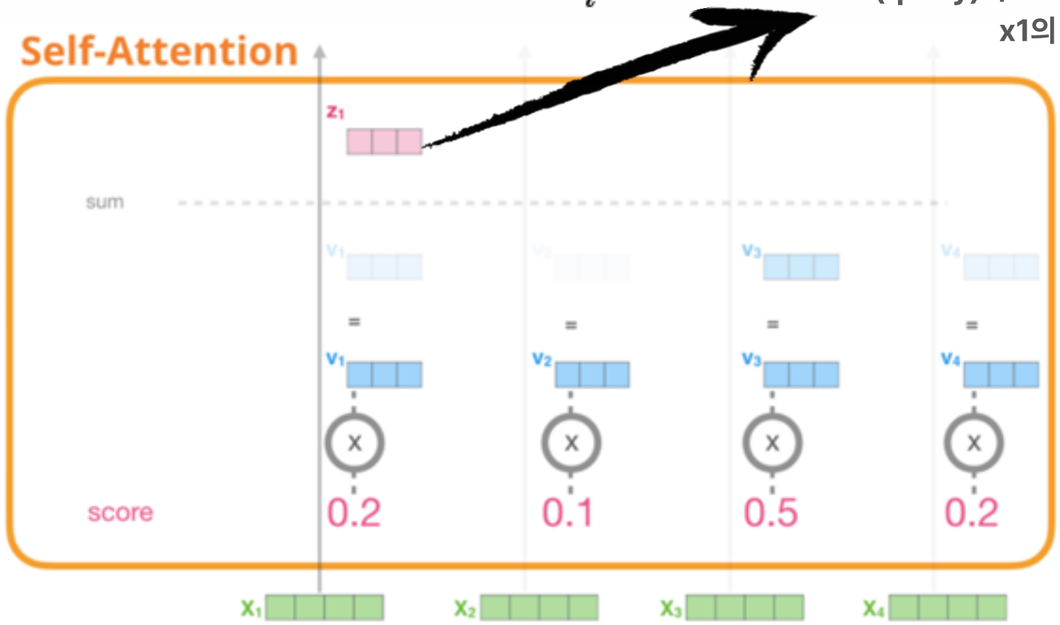
x1과 x1~xL 사이 관계도 점수 L개

Transformer: Self-Attention



$$\text{output}_i = \sum_i \alpha_{ij} v_j$$

x1(query)과 x1~xL 사이 관계가 반영된
x1의 임베딩 값



각 value vector에 대응되는 관계도 점수를 가중치로서 곱하고 sum = z1

Transformer: Self-Attention 차원

계속 강조하지만... 차원 이해하는 것은
모델 이해 및 설계에 매우 중요

L: 시퀀스 길이
d: 모델 차원

$x \in R^{L \times d}$ $W \in R^{d \times d}$ ⇨ W 차원은 이렇게 둠

$$q = W^Q x \ , k = W^k x \ , v = W^V x$$

q: d x L, W^Q: d x d, x: d x L
k: d x L, W^k: d x d, x: d x L
v: d x L, W^V: d x d, x: d x L

행과 열은
자유롭게 바꿀 수 있음
Transpose를 하거나 W와 x 위치를 바꾸거나
모델 설계자 마음 (행과 열이 어떤 의미인지는 계속 tracking해야 함)

$$e_{ij} = \frac{q_i^T k_j}{\sqrt{d}}$$

q_i^T: L x d, k_j: d x L
e_{ij}: L x L
$\sqrt{d}$: 스칼라

활성화 함수는 차원을 바꾸지 않음

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

α_{ij}: L x L

	x1	x2	...	xL
x1	관계도	관계도		관계도
x2	관계도	관계도		
...				관계도
xL	관계도	관계도	관계도	관계도

$$\text{output}_i = \sum_i \alpha_{ij} v_j$$

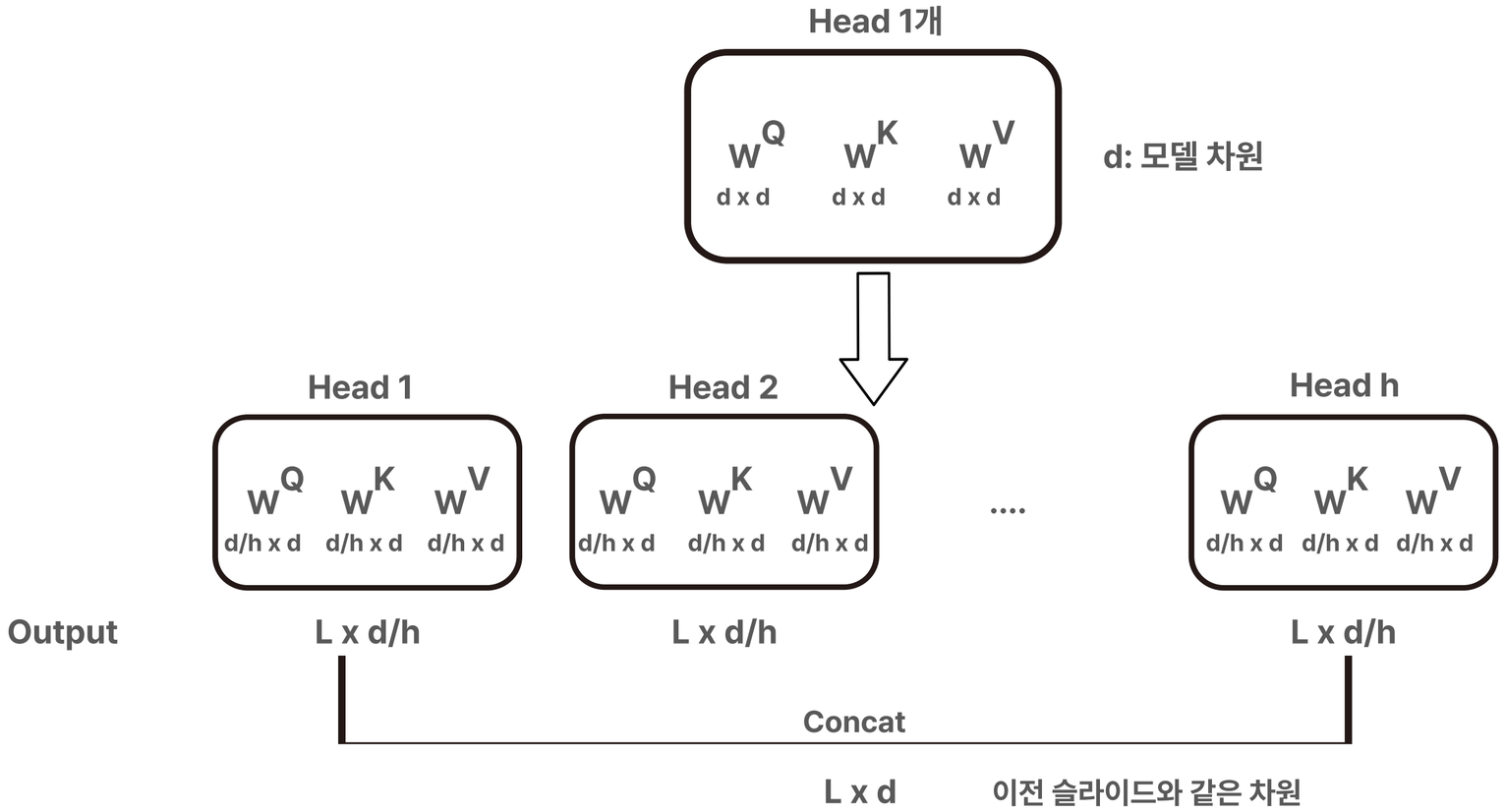
α_{ij}: L x L, v_j: L x d
output: L x d

z = [z1, ..., zL]

Transformer: Multi-Head Attention

Transformer의 attention 연산은 연산이 일어나는 head를 여러 개 두어 설계

이득 1) 각 GPU에 여러 attention head를 나눠 할당하면 병렬 연산이 가능 (분산학습)
이득 2) 각 head별로 고유의 파라미터를 두면, 각 head가 인풋 sequence 내 토큰 간의 관계를 서로 다른 관점에서 파악 → 모델의 표현력 증가



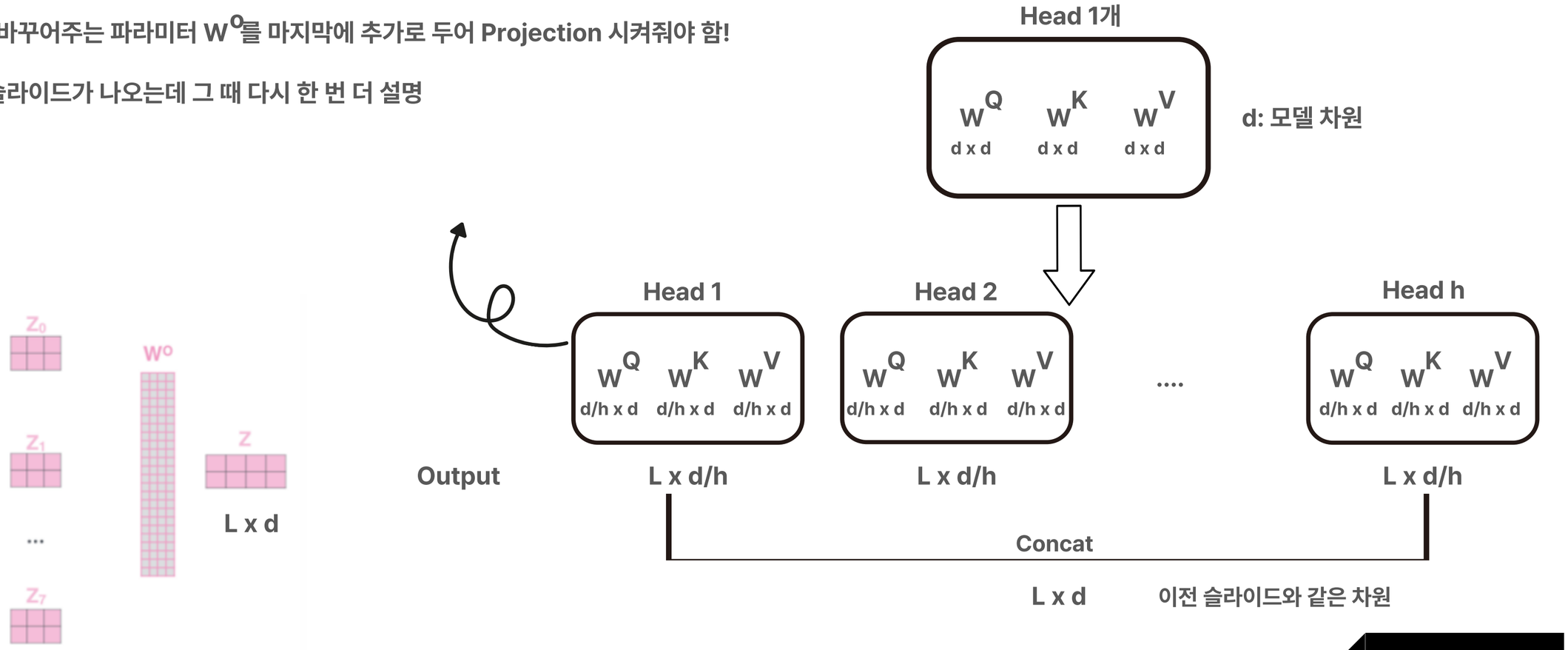
Transformer: Multi-Head Attention

Transformer의 attention 연산은 연산이 일어나는 head를 여러 개 두어 설계

d/h x d에서 d/h를 아무 값이나 설정해도 됨. 다만, 그렇게 되면 최종 output의 차원이 L x d가 아니라

서 차원을 L x d로 바꾸어주는 파라미터 W^O를 마지막에 추가로 두어 Projection 시켜줘야 함!

나중에 연습 문제 슬라이드가 나오는데 그 때 다시 한 번 더 설명



Transformer: Residual Connection + Layer Normalization

Vanishing Grad 방지 + 입출력 분포 안정화

2주차 스터디 참고

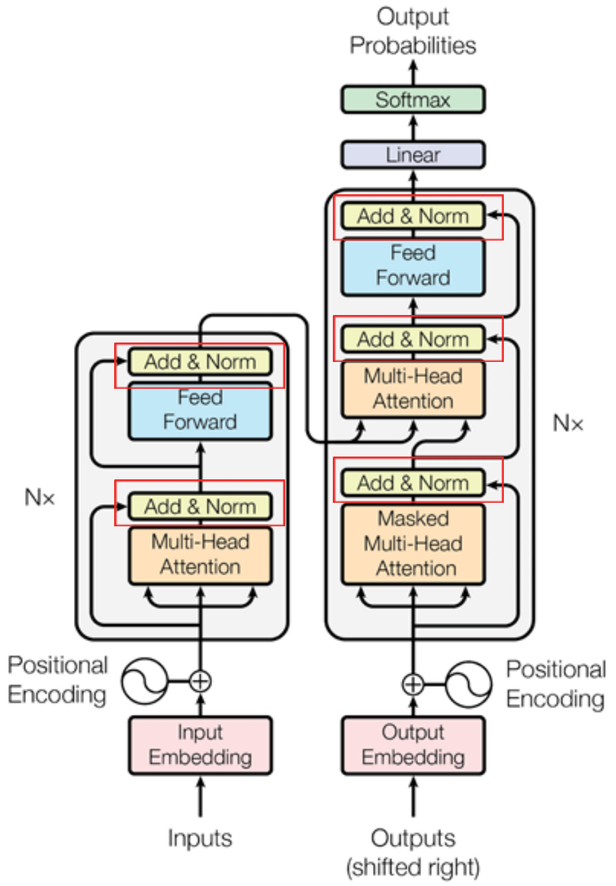
잔차연결 (Residual Connection)

$$x_{L+1} = x_L + f(x_L)$$

최종 output　f의 input　output

input을 output에 더해주는 연산

이 둘의 차원이 같아야 함 (L x d로 output 차원을 반드시 맞춰야 하는 이유)



Transformer: Residual Connection + Layer Normalization

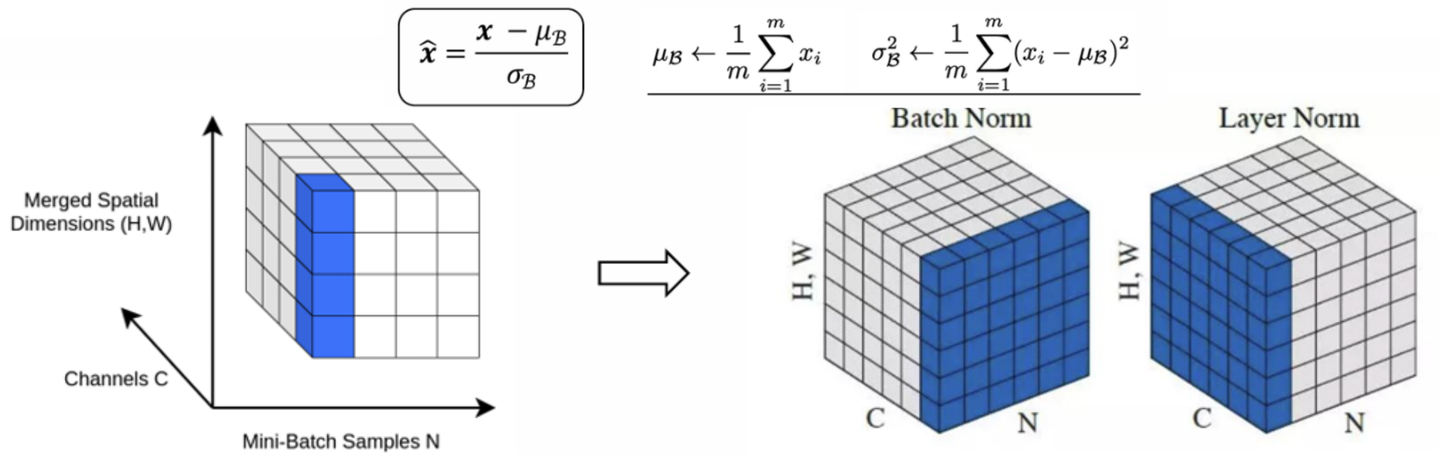
Vanishing Grad 방지 + 입출력 분포 안정화

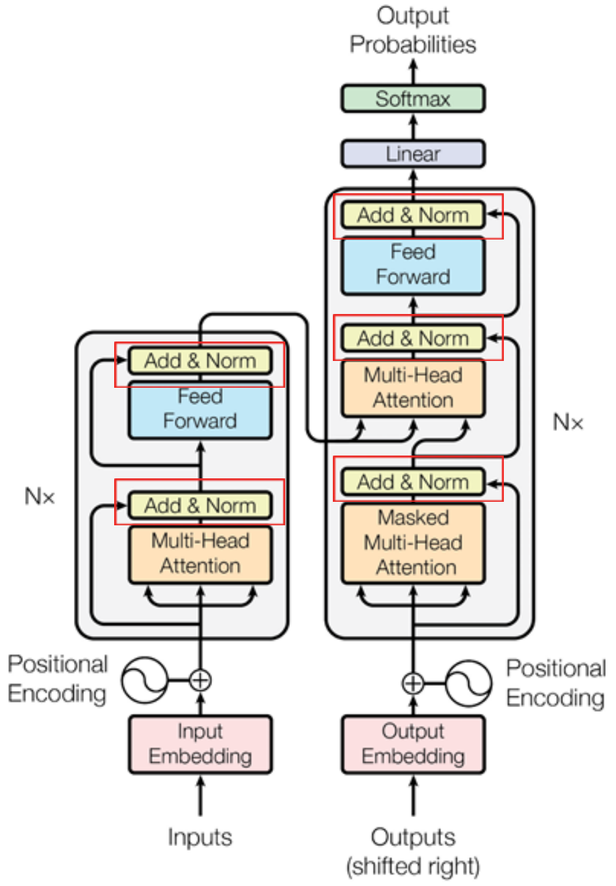
2주차 스터디 참고

Layer Normalization (Batch Normalization x)

$$\hat{x} = \frac{x - \mu_B}{\sigma_B}$$

$$\mu_B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma_B^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$





Transformer: Residual Connection + Layer Normalization

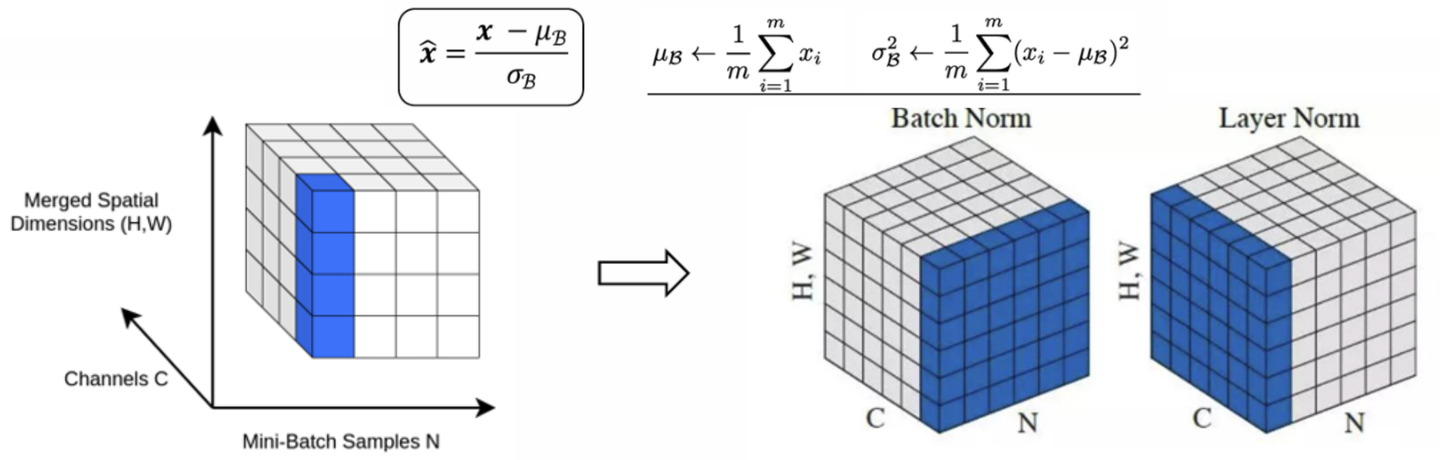
Vanishing Grad 방지 + 입출력 분포 안정화

2주차 스터디 참고

Layer Normalization (Batch Normalization x)

$$\hat{x} = \frac{x - \mu_B}{\sigma_B}$$

$$\mu_B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma_B^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

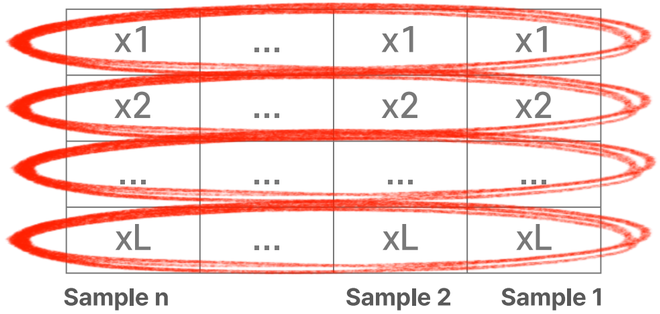


한 샘플(문장) 내 각 구성 토큰에서 차원 레벨로 normalization 수행

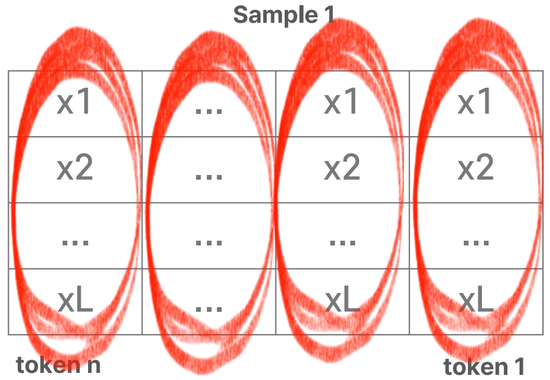
→ Across batches가 아니라 한 token 내에 있는 차원 레벨에서 정규화하는 것이기 때문에 batch size가 보통 작은 sequence data에서 batch 구성 샘플에 따라 값이 달라지지 않음

→ Across tokens가 아니라서 문장 길이에 따라 값이 달라지지 않음

One batch size = n



Batch Norm

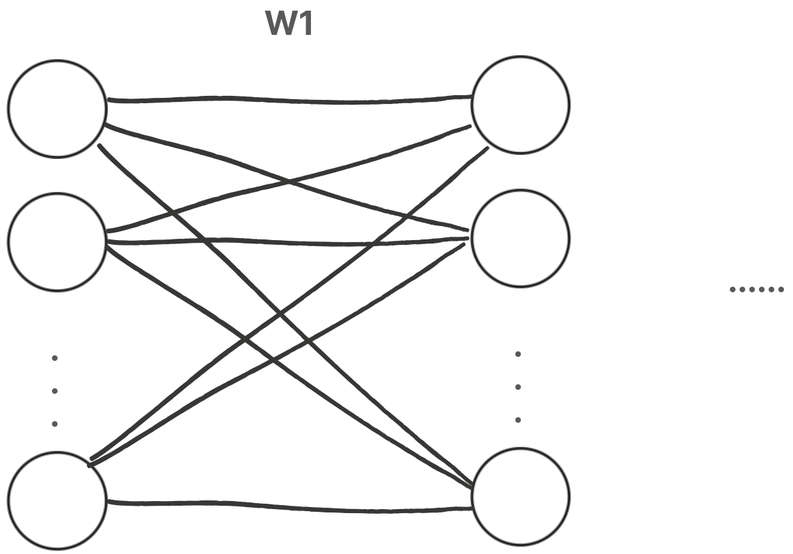


Layer Norm

L차원 token

Transformer: Feed Forward Network (FFN)

Attention을 거친 임베딩들에 담긴 혼합 정보를 이해하는 layer



Linear Layer + non-linear function

Transformer: Encoder block 차원 계산 연습 문제

이 문제를 풀 수 있으면 transformer encoder block을 확실히 이해한 것

연세대학교 첨단컴퓨팅학부 전공 수업 자연어처리 기말고사 시험 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

자연어처리 수강했을 때 기말고사에서 풀었던 문제...

문제가 되게 인상 깊었는데, 시험 끝나고 풀이를 올려주셔서 복습 용도로 사용 중...

해당 configuration을 갖춘 encoder block 1개의

전체 학습 파라미터 개수를 계산하는 문제

Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

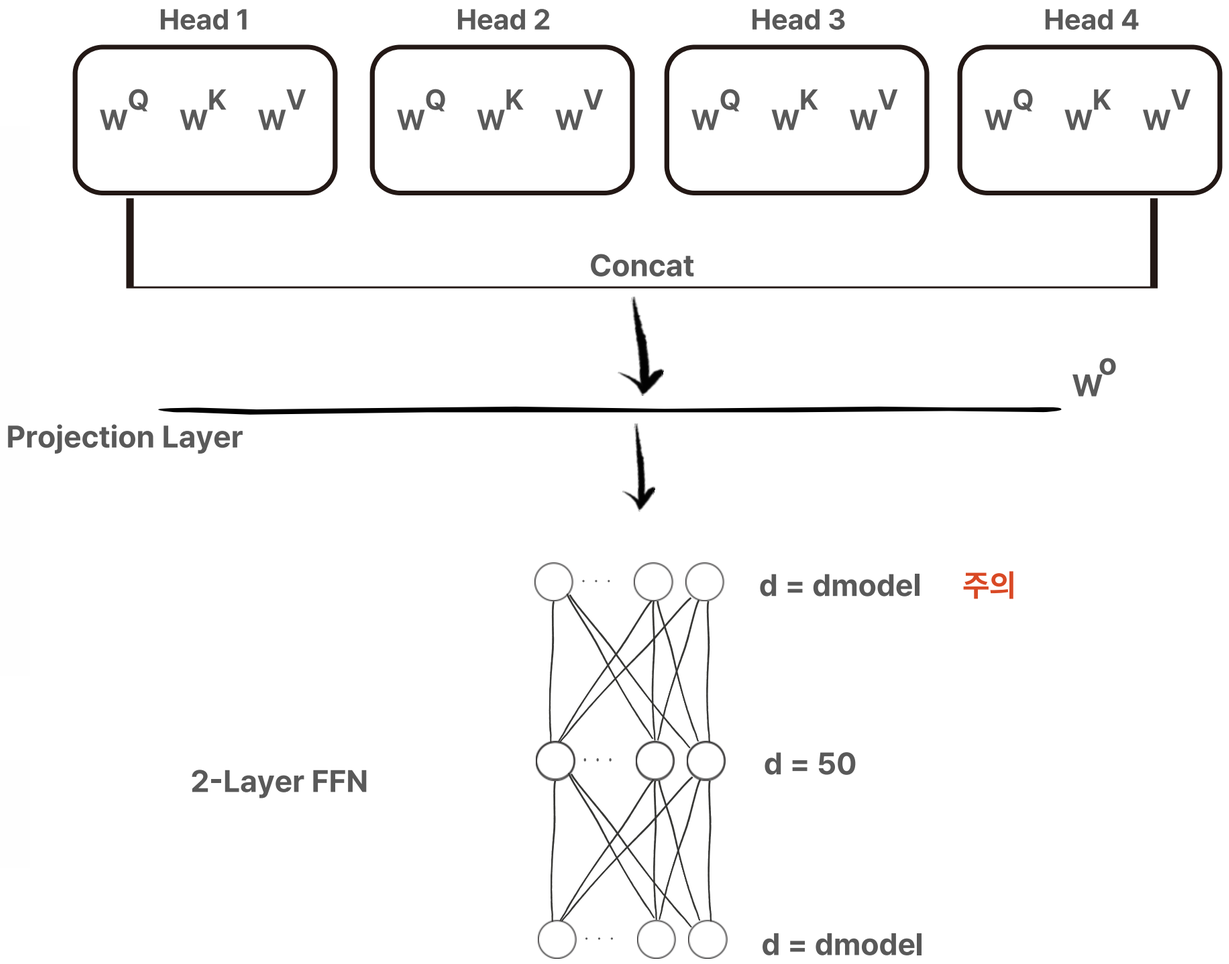
- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

주의

second projects back to d_{model}

back이라는 말 때문에
FFN의 input은 dmodel 차원



Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

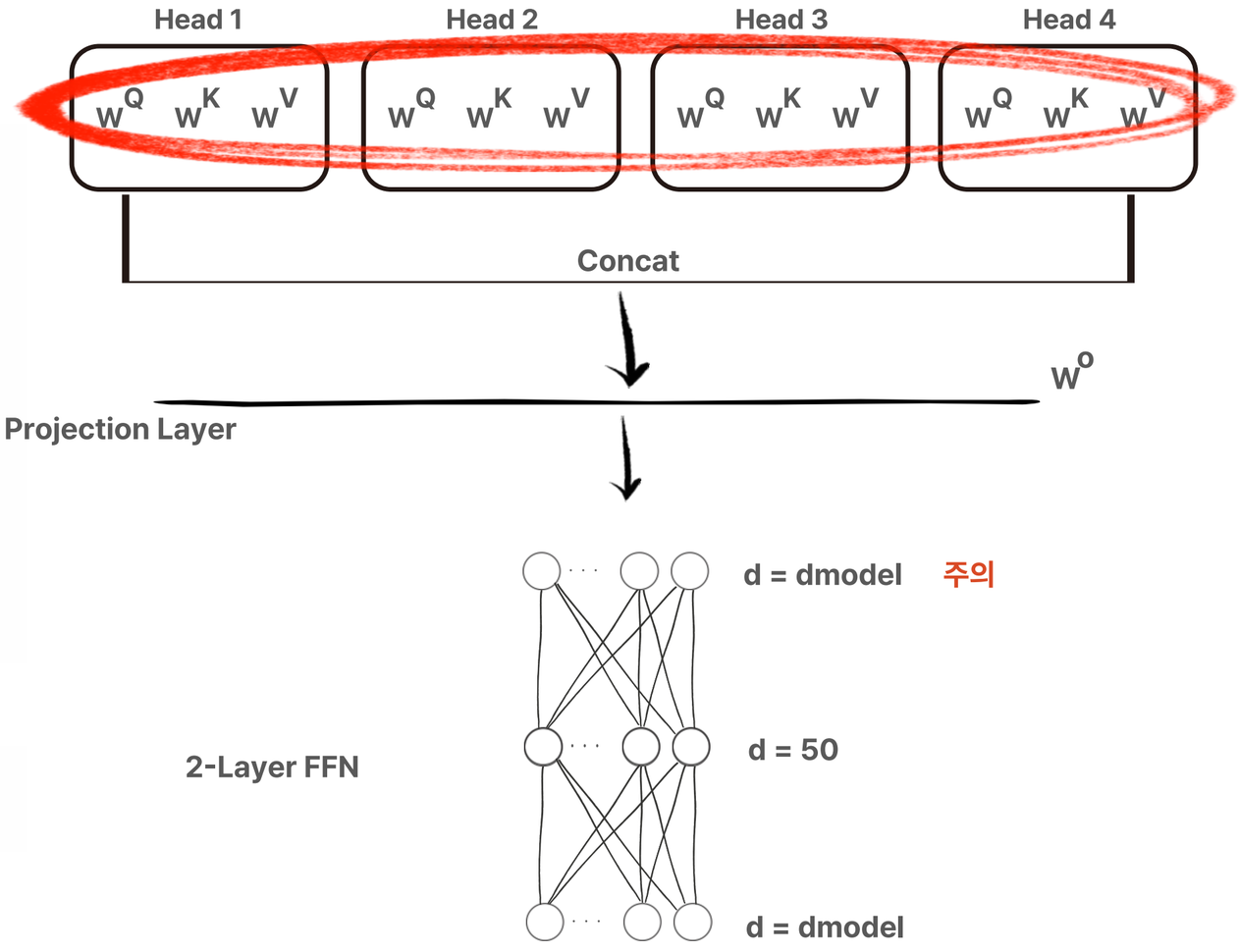
(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

$$q = W^Q x$$

dq x L ? x ? dmodel x L

따라서, ? x ? = dq x dmodel = 5 x 10

q차원이 dmodel/h가 아님!! (대신 projection layer로 차원을 맞춰줘야 함)



Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

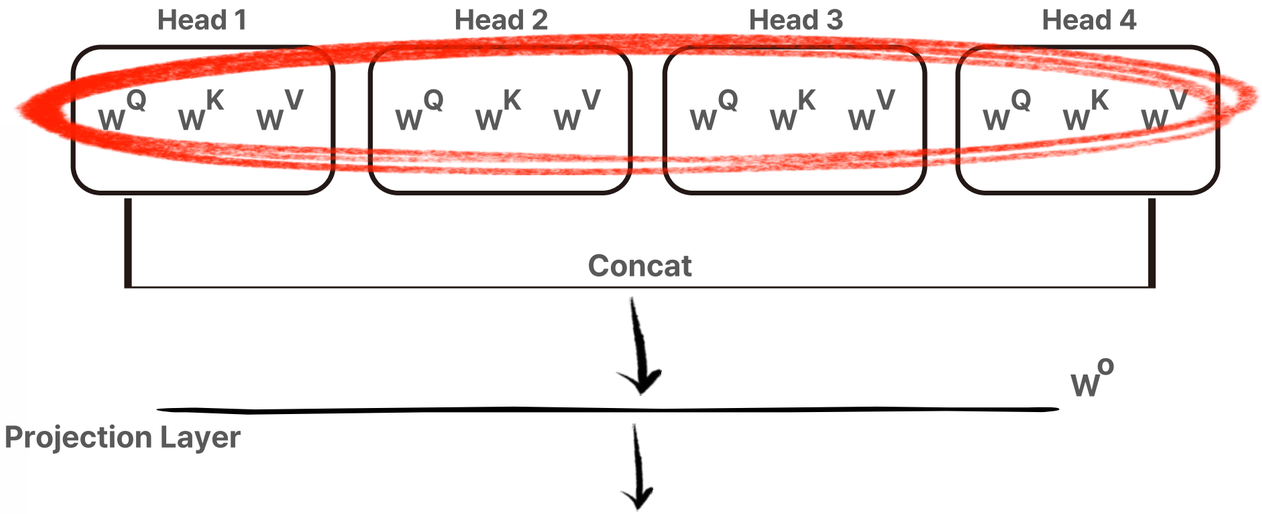
(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

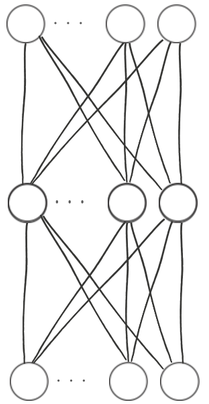
$$k = W^k x$$

dk x L = ? x ? dmodel x L

따라서, ? x ? = dk x dmodel = 5 x 10

k차원이 dmodel/h가 아님!! (대신 projection layer로 차원을 맞춰줘야 함)





2-Layer FFN

d = dmodel 주의

d = 50

d = dmodel

Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

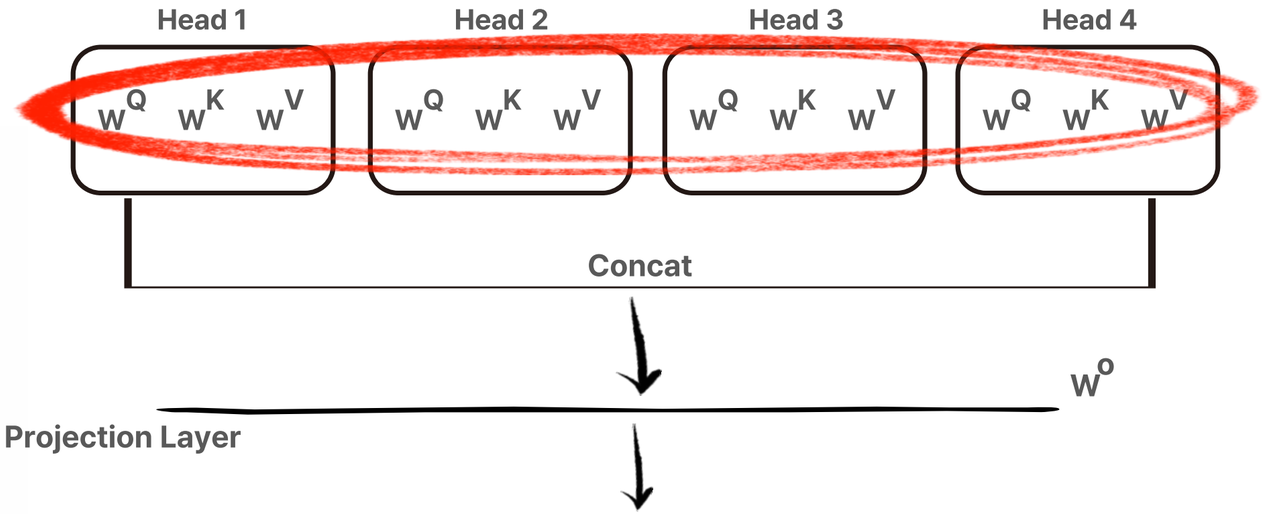
(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

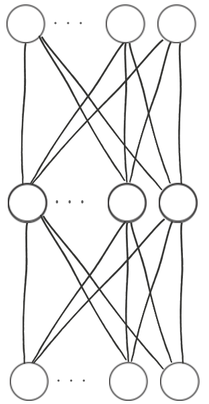
$$v = W^V x$$

dv x L ? x ? dmodel x L

따라서, ? x ? = dv x dmodel = 5 x 10

v차원이 dmodel/h가 아님!! (대신 projection layer로 차원을 맞춰줘야 함)





2-Layer FFN

d = dmodel 주의

d = 50

d = dmodel

Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

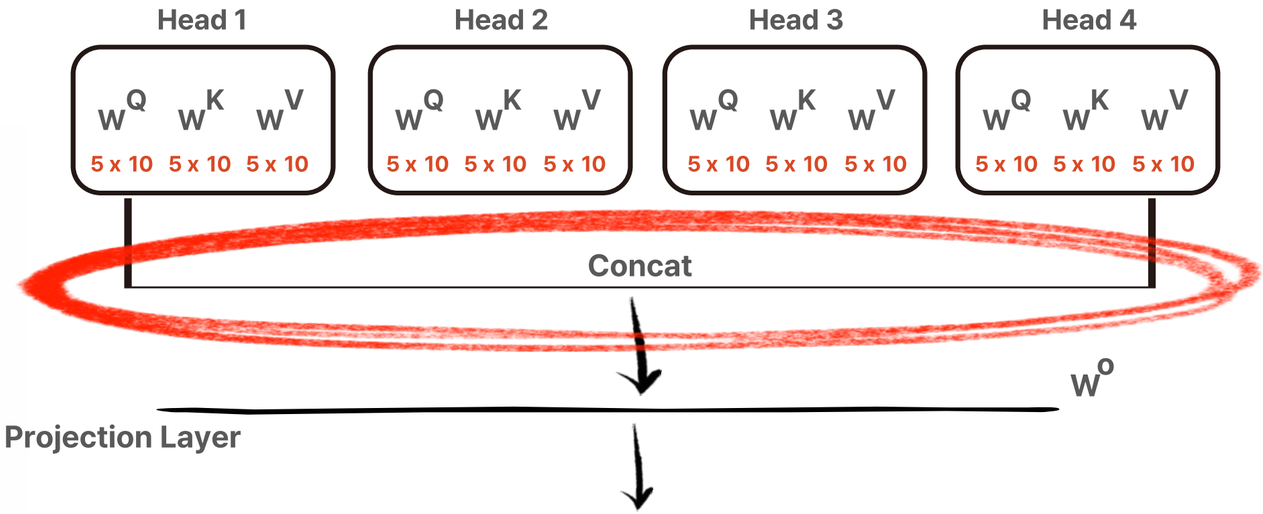
- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

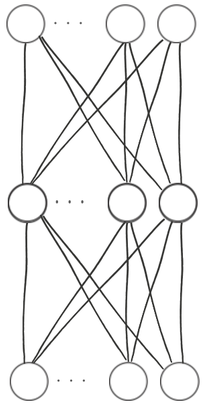
L x 5 5 x L
L x dq dk x L

$$e_{ij} = \frac{q_i^T k_j}{\sqrt{d}} \Rightarrow \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})} \Rightarrow \text{output}_i = \sum_i \alpha_{ij} v_j$$

L x L L x L L x 5 L x L L x dv L x 5



2-Layer FFN



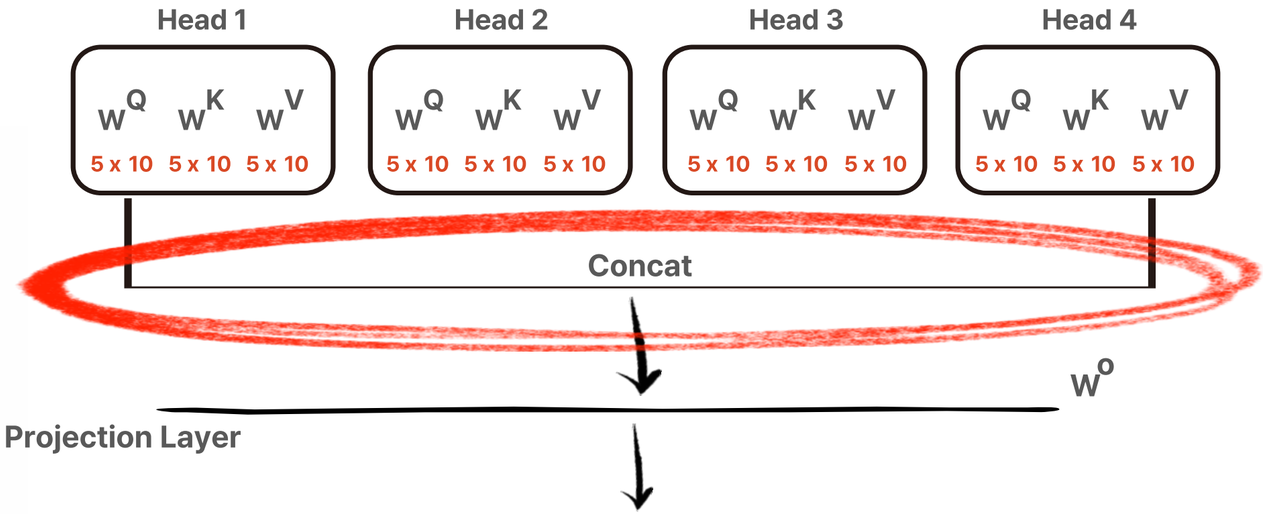
주의

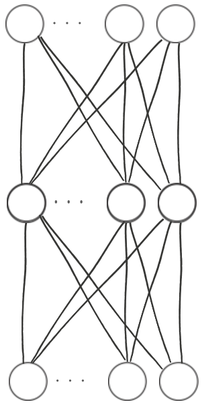
Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)





주의

2-Layer FFN

head 1개의 결과

$$\text{output}_i = \sum_i \alpha_{ij} v_j$$

L x 5

Concat
(length 차원이 아닌 차원 기준으로)

L x 20 (세로 방향으로 붙임)

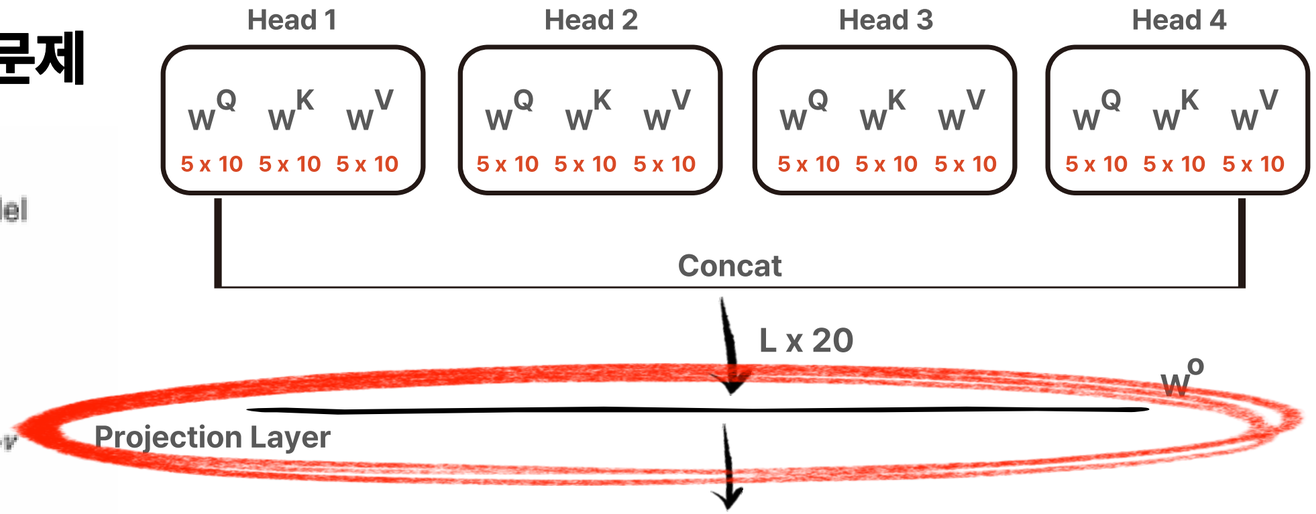
만약 q, k, v의 차원이 dmodel/h였다면 L x dmodel = L x 10

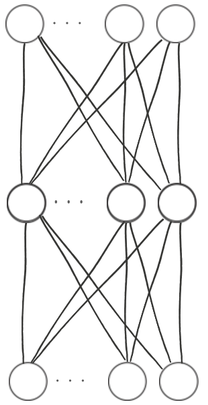
Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)





2-Layer FFN

d = dmodel 주의

d = 50

d = dmodel

W^O

L x 20 ⇨ L x dmodel = L x 10

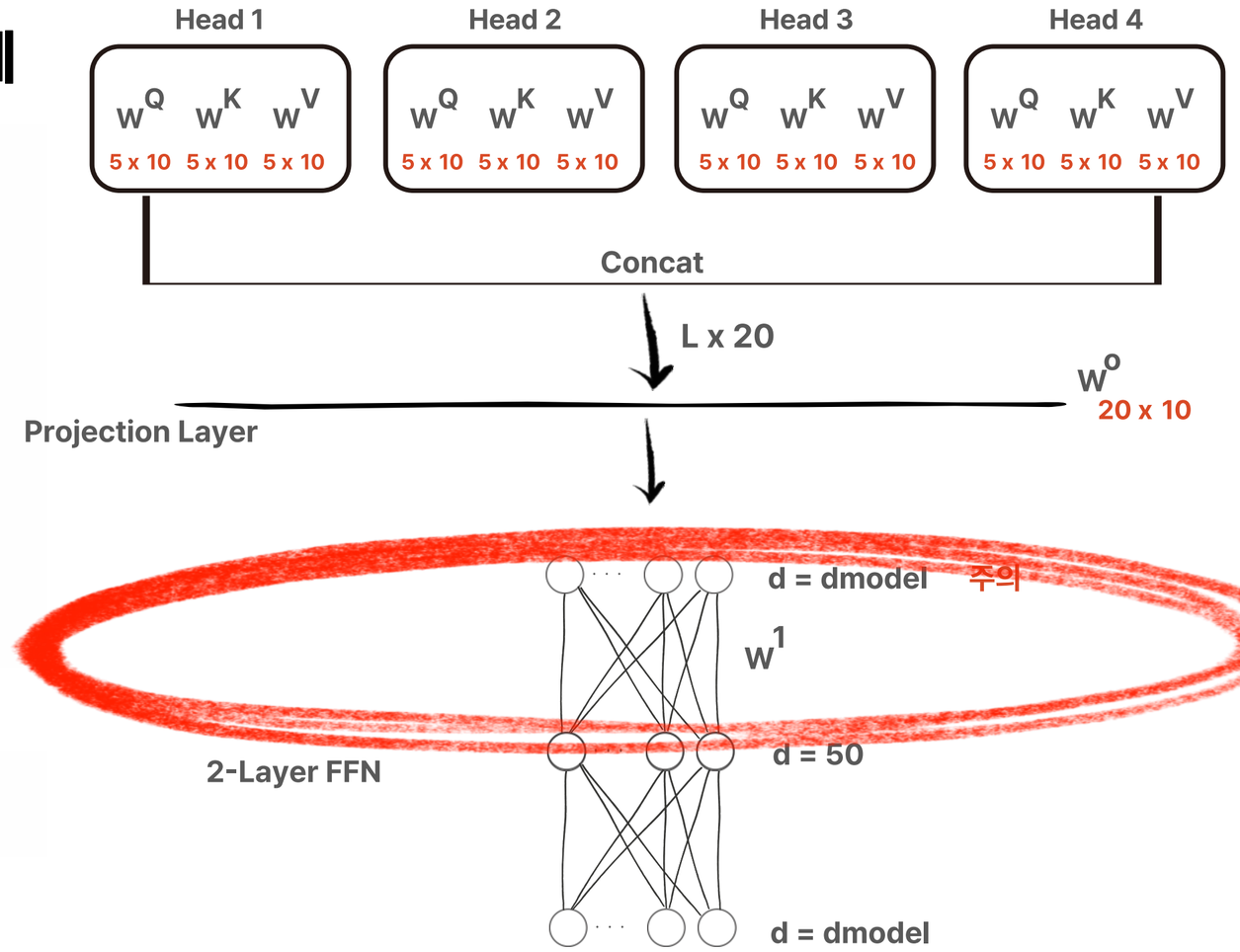
$\therefore\ W^O \in R^{20 \times 10}$

Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)



W^1

L x 10 ⇨ L x 50

$\therefore \; W^1 \in R^{10 \times 50}$

Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

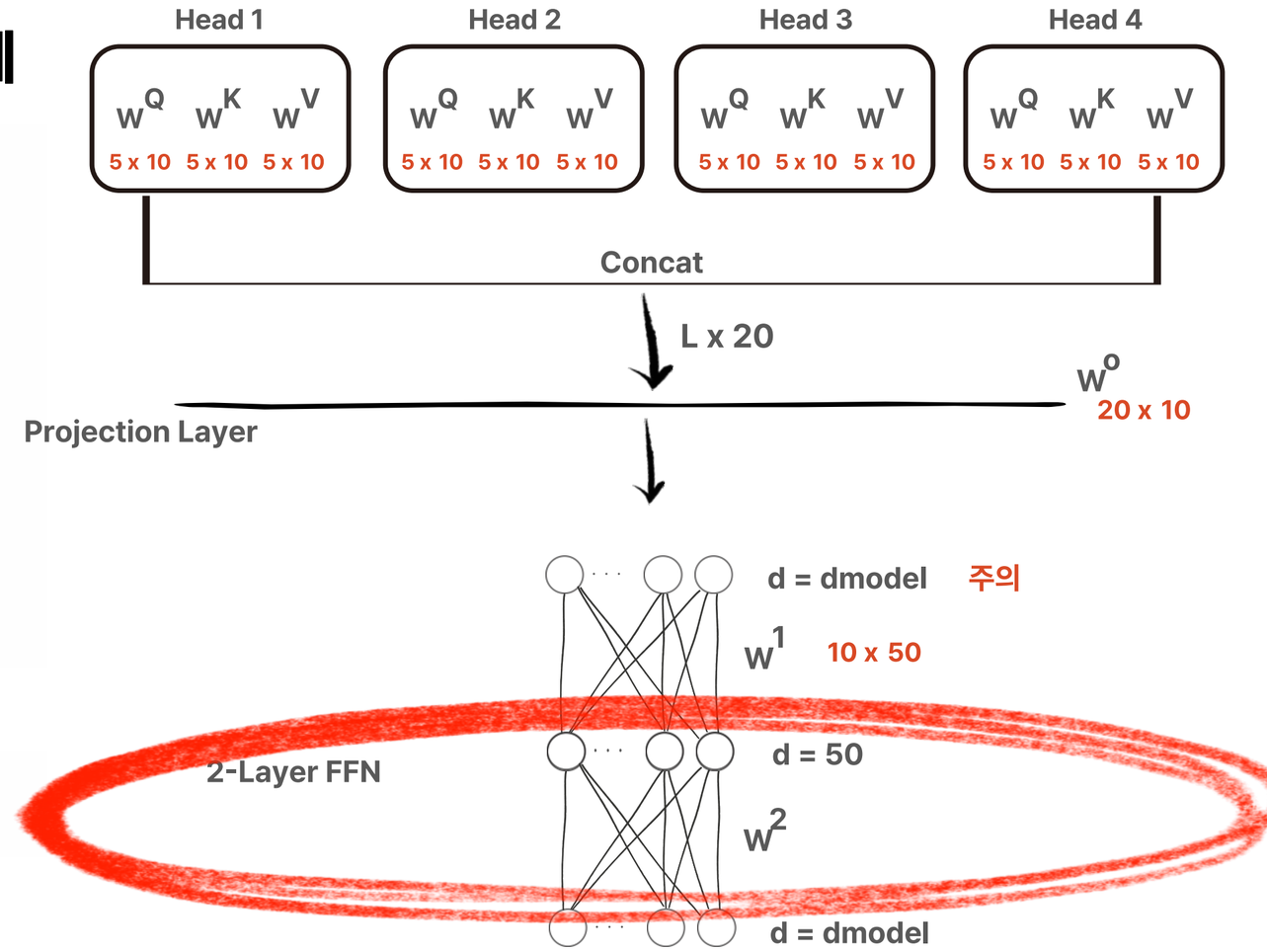
- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)

W^2

L x 50 ⇨ L x 10

$\therefore \; W^2 \in R^{50 \times 10}$

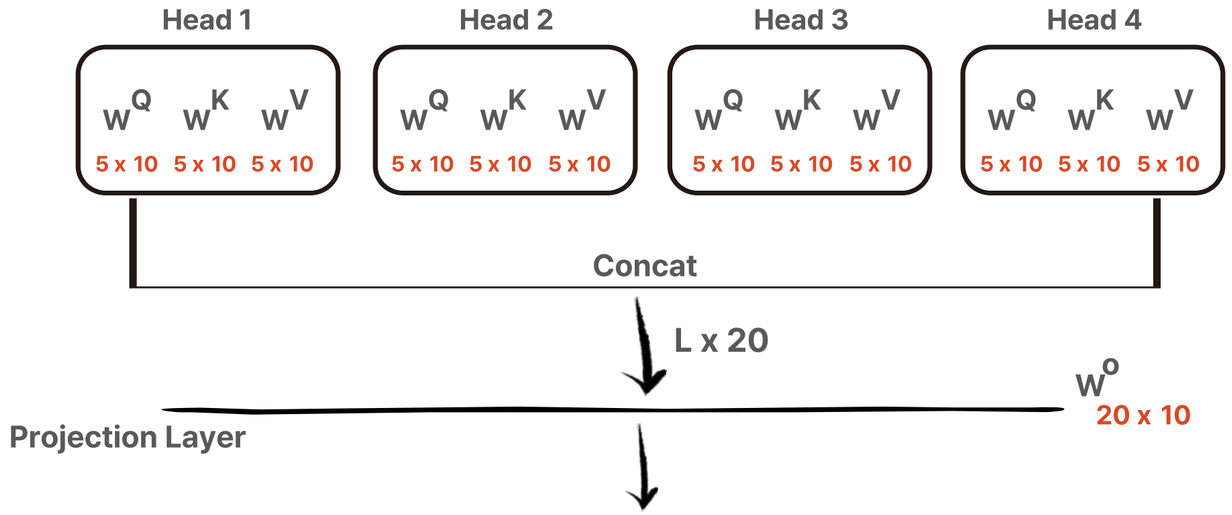


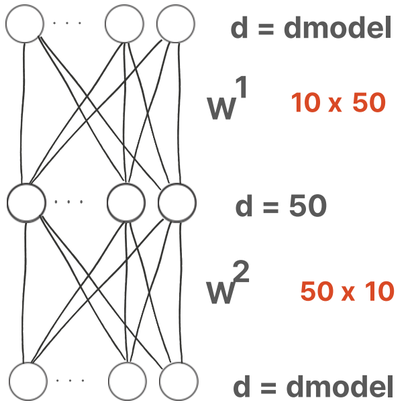
Transformer: Encoder block 차원 계산 연습 문제

[Attention and Transformer]
Q3. (15 points total) Consider a single encoder block in a Transformer model with the following configuration:

- Hidden dimension $d_{model} = 10$
- Number of attention heads $h = 4$
- Each head uses key, query, and value dimensions $d_q = d_k = d_v = 5$
- Each head has its own independent projection matrices W^Q, W^K, W^V
- A standard output projection W^O is used after concatenating head outputs
- The feedforward sublayer consists of two linear layers:
- First expands to $d_{ff} = 50$, second projects back to d_{model}
- No bias terms are used in any of the layers
- Context length = 20 tokens

(a) Calculate the total number of trainable parameters in this encoder block. (5 points)





2-Layer FFN

Trainable Parameter

W^Q W^K W^V x 4

W^O

W^1

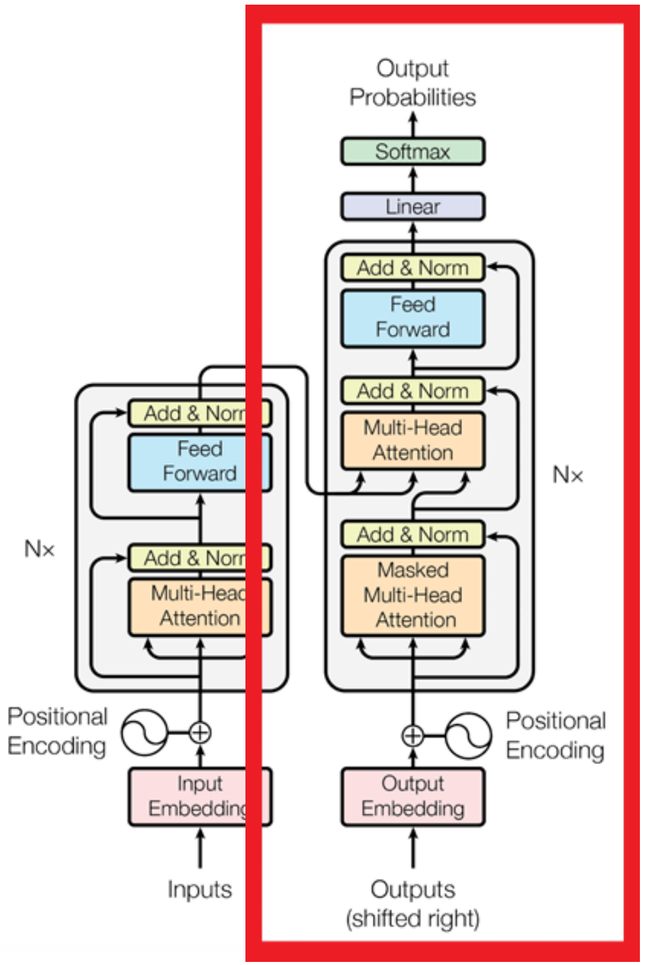
W^2

# of trainable parameter

= (5 x 10 + 5 x 10 + 5 x 10) x 4 + 20 x 10 + 10 x 50 + 50 x 10 = 1800

W^Q W^K W^V x 4 W^O W^1 W^2

Transformer Decoder



Attention Pattern 제외하고는 encoder와 원리 동일

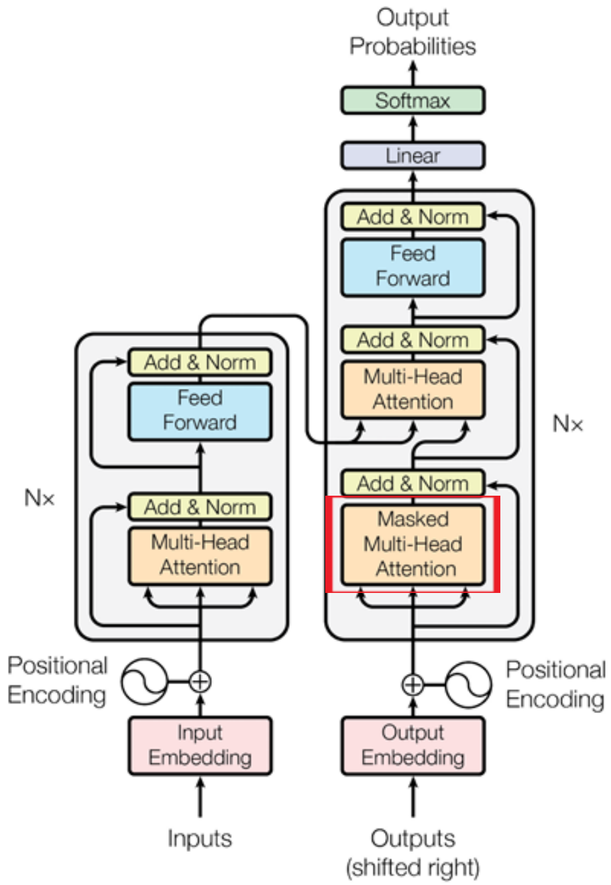
Decoder가 input을 받으면 encoder의 output(context vector)과 함께 고려하여

토큰 하나를 생성하고 생성한 토큰이 다시 decoder의 input으로 들어감.

이렇게 동일 과정을 거쳐 output sequence y = [y1, ..., yn]을 생성

(y1, y2, ... 이렇게 순차 생성하는 방식을 autoregressive 방식이라고 부름)

Transformer Decoder: Masked Multi-Head Attention



Decoder는 sequence를 생성(generation)해야함

즉, 뒤에 토큰이 뭐가 올 지 모르는 상황에서 앞에 나온 토큰들 정보만을 바탕으로 토큰을 생성해야함(left to right)

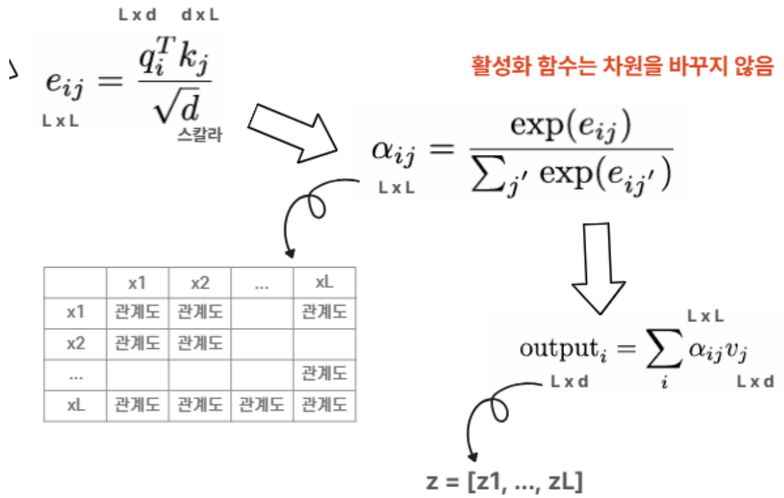
Decoder의 attention 연산은 타깃 토큰보다 뒤에 있는 토큰과의 관계도 점수는 마스킹하고 연산함

이유1: Inference 동작 방식은 토큰을 하나씩 하나씩 생성하는 방식임. 하지만 training할 때는 전체 문장을 알고 있는 상태이기 때문에 토큰 하나 생성할 때마다 Cross Entropy를 하나씩 계산하기 보다 빠른 속도를 위해 문장 전체를 한 번에 넣고 각각의 Cross Entropy를 동시에 계산함 → 일반 attention 연산을 취하면 미래 토큰을 아는 상태로 cross entropy를 계산하는 것이라 올바르지 않음!!

이유2: Inference할 때도 문장 전체를 한 번에 input으로 받는 단계가 존재 (prefilling: 프롬프트 이해 단계)

Transformer Decoder: Masked Multi-Head Attention

따라서 attention 연산을 수행하되, 타깃 토큰보다 뒤에 있는 토큰과의 관계도 점수는 0으로 마스킹하고 연산함



	x1	x2	...	xL
x1	관계도	관계도		관계도
x2	관계도	관계도		
...				관계도
xL	관계도	관계도	관계도	관계도

학습 문장: SBBL은 최고의 Bioinformatics 연구실이다.

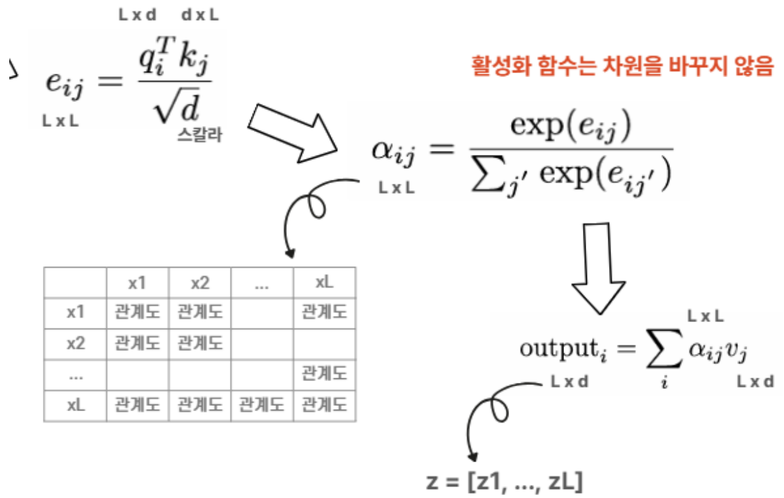
	SBBL은	시스템생물학과	최고의	Bioinformatics	연구실이다.
SBBL은	관계도	관계도	관계도	관계도	관계도
시스템생물학과	관계도	관계도	관계도	관계도	관계도
최고의	관계도	관계도	관계도	관계도	관계도
Bioinformatics	관계도	관계도	관계도	관계도	관계도
연구실이다.	관계도	관계도	관계도	관계도	관계도

이렇게 학습하면 앞 뒤 문맥을 보고 생성하는 방법을 배우는 것임

Inference 환경과 다른 task

Transformer Decoder: Masked Multi-Head Attention

따라서 attention 연산을 수행하되, 타깃 토큰보다 뒤에 있는 토큰과의 관계도 점수는 0으로 마스킹하고 연산함

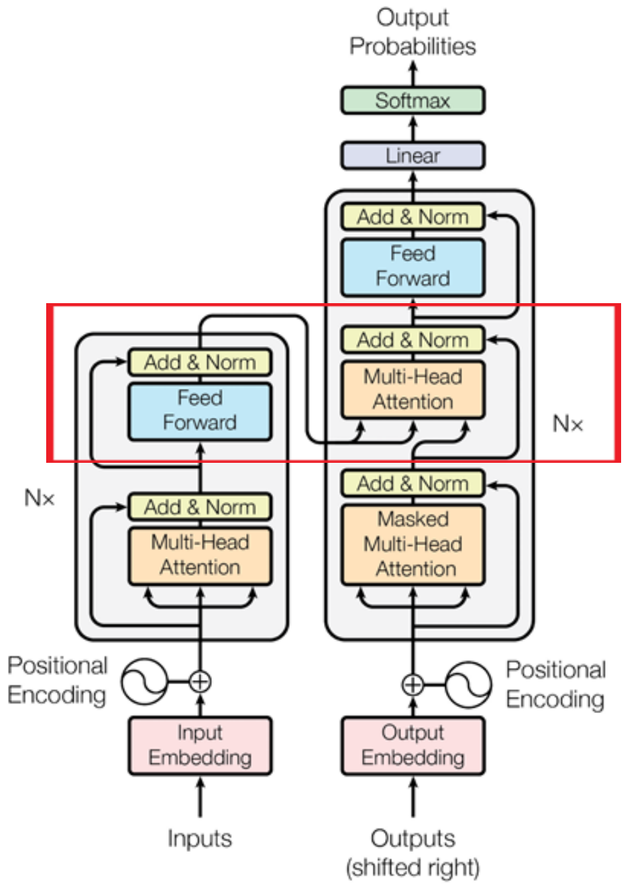


	x1	x2	...	xL
x1	관계도	관계도		관계도
x2	관계도	관계도		
...				관계도
xL	관계도	관계도	관계도	관계도

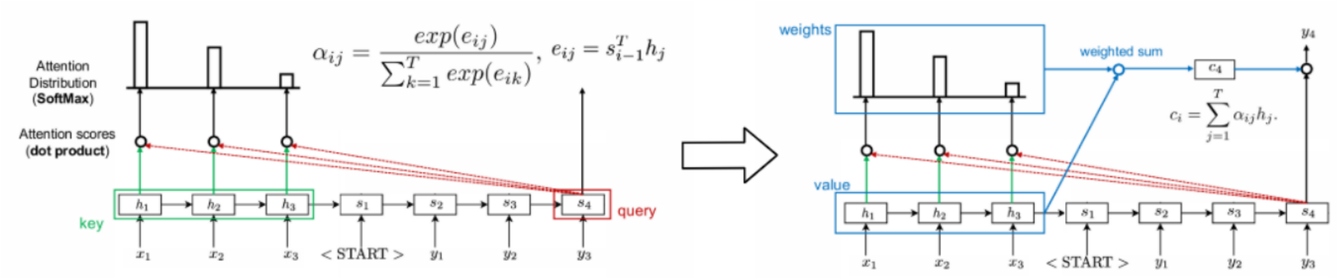
학습 문장: SBBL은 시스템생물학과 최고의 Bioinformatics 연구실이다.

	SBBL은	시스템생물학과	최고의	Bioinformatics	연구실이다.
SBBL은	관계도	0	0	0	0
시스템생물학과	관계도	관계도	0	0	0
최고의	관계도	관계도	관계도	0	0
Bioinformatics	관계도	관계도	관계도	관계도	0
연구실이다.	관계도	관계도	관계도	관계도	관계도

Transformer Decoder: Encoder-Decoder Cross Attention



Seq2Seq attention pattern이랑 완전 동일



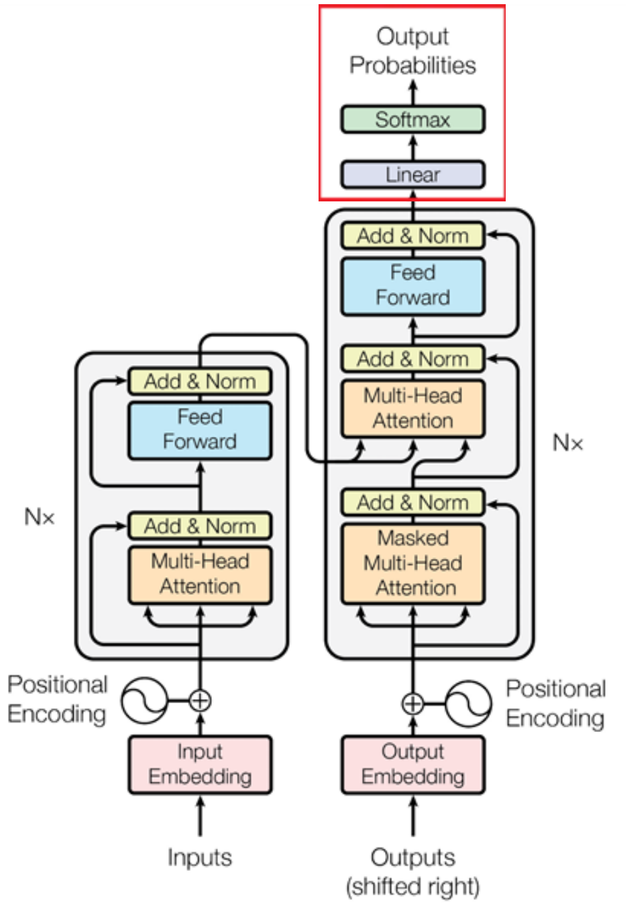
Transformer Decoder: Output Generation Head

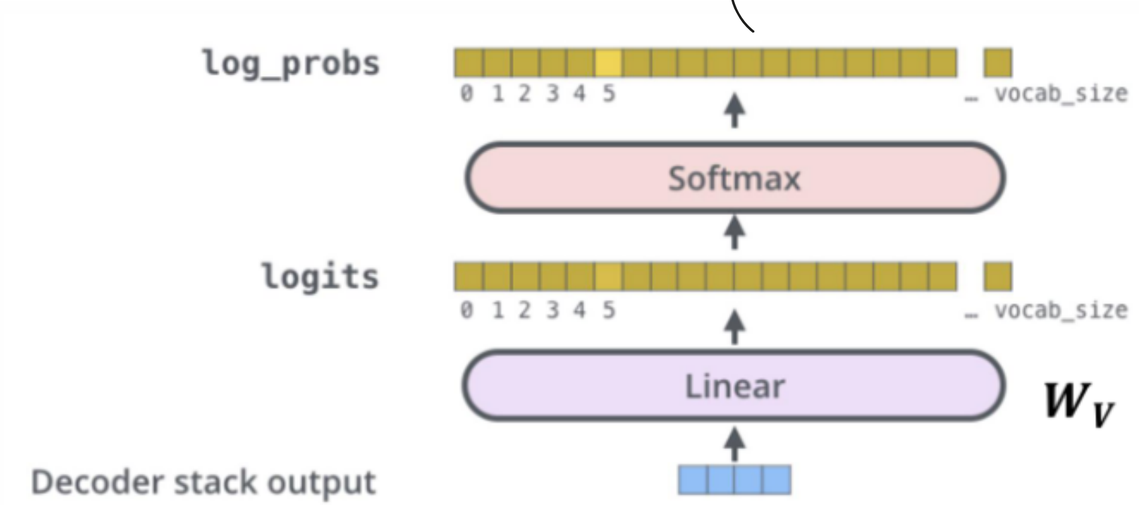
Training이면 cross-entropy 계산

Inference면 가장 높은 확률의 토큰 생성

vocab내 각 토큰들의 확률 분포

최종 output $\in R^{(vocab_size) \times 1}$





3주차 Sequence Data Learning Summary

Sequence data를 처리하기 위해 사용한 기법들 이해 → Transformer 이해에 사용됨 + 딥러닝에 익숙해지기 (ex. Forward pass, Backpropagation)

이들에게 어떤 단점이 있었기에 Transformer까지 발전할 수 있었는 지 이해하기 → Transformer 이해에 사용됨